```
Start coding or generate with AI.
```

```
##q1
```

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns

# For cleaner visual style
sns.set(style="whitegrid")
```

```
df = pd.read_csv("austinHousingData.csv", on_bad_lines='skip')
print(df.head())
```

```
         zpid          city          streetAddress  zipcode  \
0   111373431  pflugerville   14424 Lake Victor Dr    78660
1   120900430  pflugerville     1104 Strickling Dr    78660
2  2084491383  pflugerville    1408 Fort Dessau Rd    78660
3   120901374  pflugerville     1025 Strickling Dr    78660
4    60134862  pflugerville  15005 Donna Jane Loop    78660

                                      description   latitude  longitude  \
0  14424 Lake Victor Dr, Pflugerville, TX 78660 i...  30.430632 -97.663078
1  Absolutely GORGEOUS 4 Bedroom home with 2 full...  30.432673 -97.661697
2  Under construction - estimated completion in A...  30.409748 -97.639771
3  Absolutely darling one story home in charming ...  30.432112 -97.661659
4  Brimming with appeal & warm livability! Sleek ...  30.437368 -97.656860

   propertyTaxRate  garageSpaces  hasAssociation  ...  numOfMiddleSchools  \
0             1.98             2            True  ...                   1
1             1.98             2            True  ...                   1
2             1.98             0            True  ...                   1
3             1.98             2            True  ...                   1
4             1.98             0            True  ...                   1

   numOfHighSchools  avgSchoolDistance  avgSchoolRating  avgSchoolSize  \
0                 1           1.266667         2.666667           1063
1                 1           1.400000         2.666667           1063
2                 1           1.200000         3.000000           1108
3                 1           1.400000         2.666667           1063
4                 1           1.133333         4.000000           1223

  MedianStudentsPerTeacher  numOfBathrooms  numOfBedrooms  numOfStories  \
0                       14             3.0              4             2
1                       14             2.0              4             1
2                       14             2.0              3             1
3                       14             2.0              3             1
4                       14             3.0              3             2

                                       homeImage
0  111373431_ffce26843283d3365c11d81b8e6bdc6f-p_f...
1  120900430_8255c127be8dcf0a1a18b7563d987088-p_f...
2  2084491383_a2ad649e1a7a098111dcea084a11c855-p_...
3  120901374_b469367a619da85b1f5ceb69b675d88e-p_f...
4  60134862_b1a48a3df3f111e005bb913873e98ce2-p_f.jpg

[5 rows x 47 columns]
```

```
# BASIC DATA EXPLORATION

print("Shape of dataset:", df.shape)
print("\nColumn names:\n", df.columns)

print("\nData Types:\n")
print(df.dtypes)

print("\nSummary Statistics:\n")
df.describe(include='all')
```

```
Shape of dataset: (15171, 47)

Column names:
 Index(['zpid', 'city', 'streetAddress', 'zipcode', 'description', 'latitude',
       'longitude', 'propertyTaxRate', 'garageSpaces', 'hasAssociation',
       'hasCooling', 'hasGarage', 'hasHeating', 'hasSpa', 'hasView',
       'homeType', 'parkingSpaces', 'yearBuilt', 'latestPrice',
       'numPriceChanges', 'latest_saledate', 'latest_salemonth',
       'latest_saleyear', 'latestPriceSource', 'numOfPhotos',
       'numOfAccessibilityFeatures', 'numOfAppliances', 'numOfParkingFeatures',
       'numOfPatioAndPorchFeatures', 'numOfSecurityFeatures',
       'numOfWaterfrontFeatures', 'numOfWindowFeatures',
       'numOfCommunityFeatures', 'lotSizeSqFt', 'livingAreaSqFt',
       'numOfPrimarySchools', 'numOfElementarySchools', 'numOfMiddleSchools',
       'numOfHighSchools', 'avgSchoolDistance', 'avgSchoolRating',
       'avgSchoolSize', 'MedianStudentsPerTeacher', 'numOfBathrooms',
       'numOfBedrooms', 'numOfStories', 'homeImage'],
      dtype='object')
```

```
# MISSING VALUE ANALYSIS

missing = df.isnull().sum()
print("Missing Values:\n")
print(missing)

plt.figure(figsize=(10,5))
sns.heatmap(df.isnull(), cbar=False, cmap='viridis')
plt.title("Missing Value Heatmap")
plt.show()
```

```
garageSpaces                int64
hasAssociation               bool
hasCooling                   bool
hasGarage                    bool
hasHeating                   bool
hasSpa                       bool
hasView                      bool
homeType                   object
parkingSpaces               int64
yearBuilt                   int64
latestPrice               float64
numPriceChanges             int64
latest_saledate            object
latest_salemonth            int64
latest_saleyear             int64
latestPriceSource          object
numOfPhotos                 int64
numOfAccessibilityFeatures  int64
numOfAppliances             int64
numOfParkingFeatures        int64
numOfPatioAndPorchFeatures  int64
numOfSecurityFeatures       int64
numOfWaterfrontFeatures     int64
numOfWindowFeatures         int64
numOfCommunityFeatures      int64
lotSizeSqFt               float64
livingAreaSqFt            float64
numOfPrimarySchools         int64
numOfElementarySchools      int64
numOfMiddleSchools          int64
numOfHighSchools            int64
avgSchoolDistance         float64
avgSchoolRating           float64
avgSchoolSize               int64
MedianStudentsPerTeacher    int64
numOfBathrooms            float64
numOfBedrooms               int64
numOfStories                int64
homeImage                  object
dtype: object

Summary Statistics:
```

	zpid	city	streetAddress	zipcode	description	latitude	longitude	propertyTaxRate	garageSpaces
count	1.517100e+04	15171	15171	15171.000000	15169	15171.000000	15171.000000	15171.000000	15171.000000
unique	NaN	9	15164	NaN	15132	NaN	NaN	NaN	NaN
top	NaN	austin	2404 Independence Dr	NaN	Coming soon! Photos and details will be availa...	NaN	NaN	NaN	NaN
freq	NaN	15020	2	NaN	12	NaN	NaN	NaN	NaN

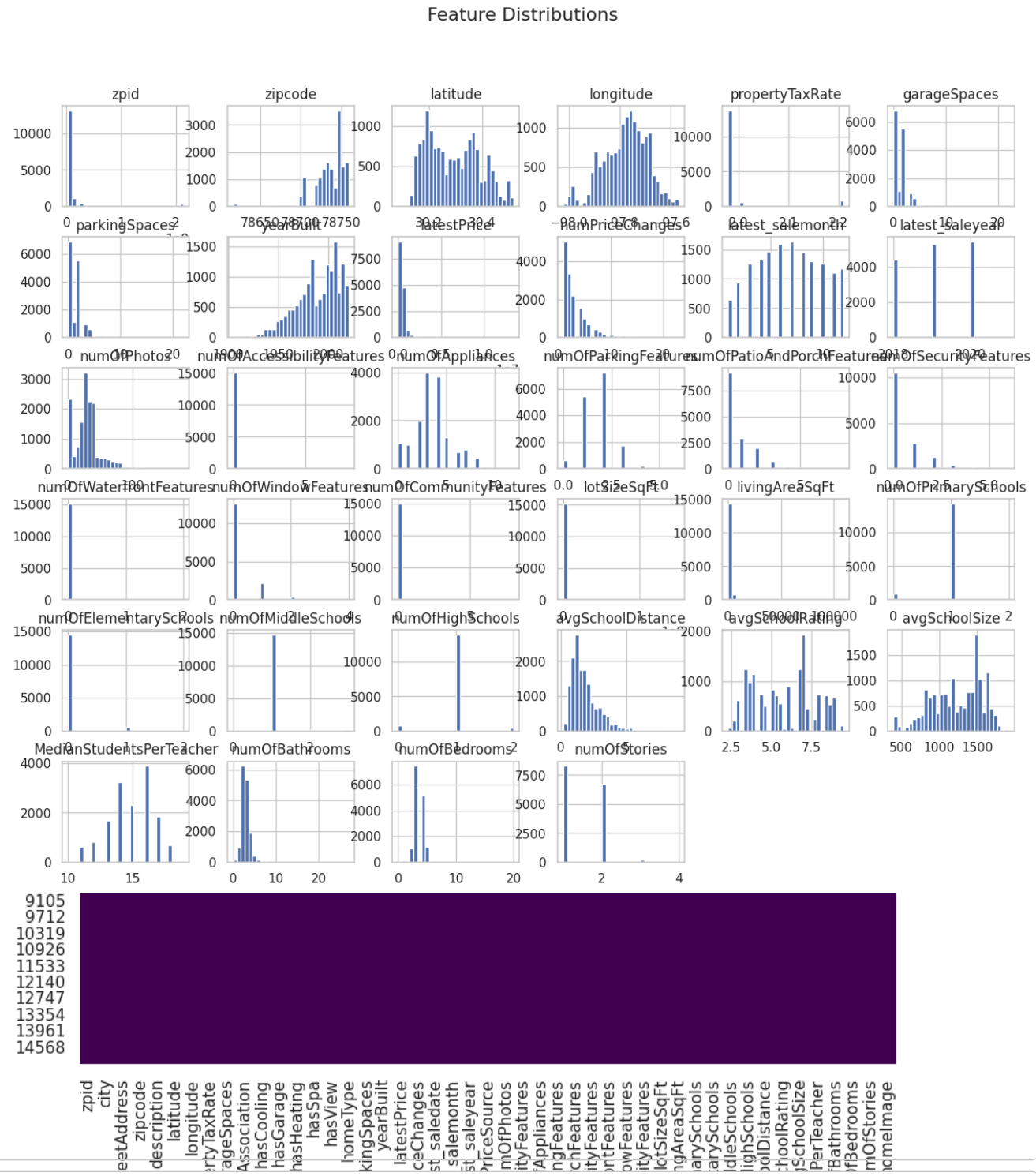
mean	1.044193e+08	NaN	NaN	78735.932964	NaN	30.291596	-97.778532	1.994085	1.229187
std	3.179426e+08	NaN	NaN	18.893475	NaN	0.096973	0.084715	0.053102	1.352117
min	2.858495e+07	NaN	NaN	78617.000000	NaN	30.085030	-98.022057	1.980000	0.000000
25%	2.941115e+07	NaN	NaN	78727.000000	NaN	30.203313	-97.838009	1.980000	0.000000
50%	2.949441e+07	NaN	NaN	78739.000000	NaN	30.284416	-97.769539	1.980000	1.000000
75%	7.033762e+07	NaN	NaN	78749.000000	NaN	30.366585	-97.717903	1.980000	2.000000
max	2.146313e+09	NaN	NaN	78759.000000	NaN	30.517323	-97.569504	2.210000	22.000000

11 rows × 47 columns

```
Missing Values:

zpid                          0
city                          0
```

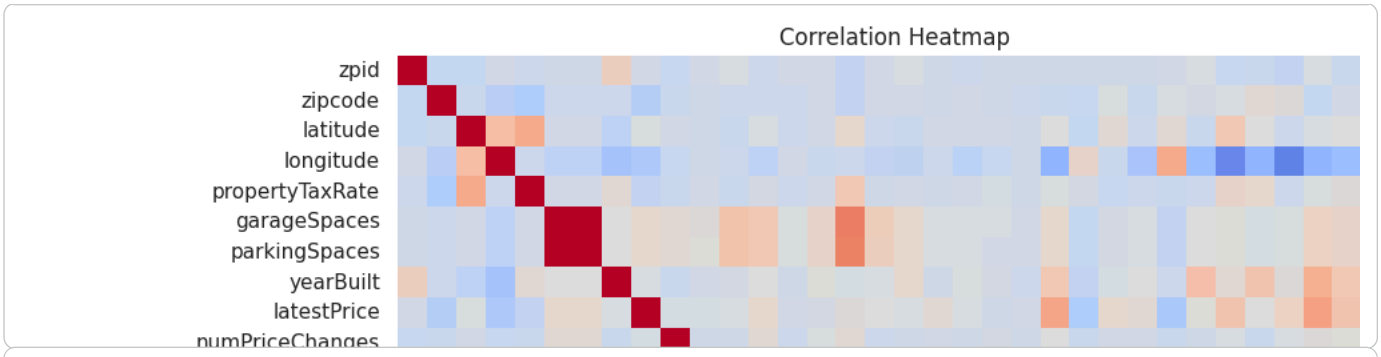
```
# FEATURE DISTRIBUTIONS
#    (Histograms for numerical variables)

numeric_cols = df.select_dtypes(include=[np.number]).columns

df[numeric_cols].hist(figsize=(15, 12), bins=30)
plt.suptitle("Feature Distributions", fontsize=16)
plt.show()
```

```
hasSpa                        0
```



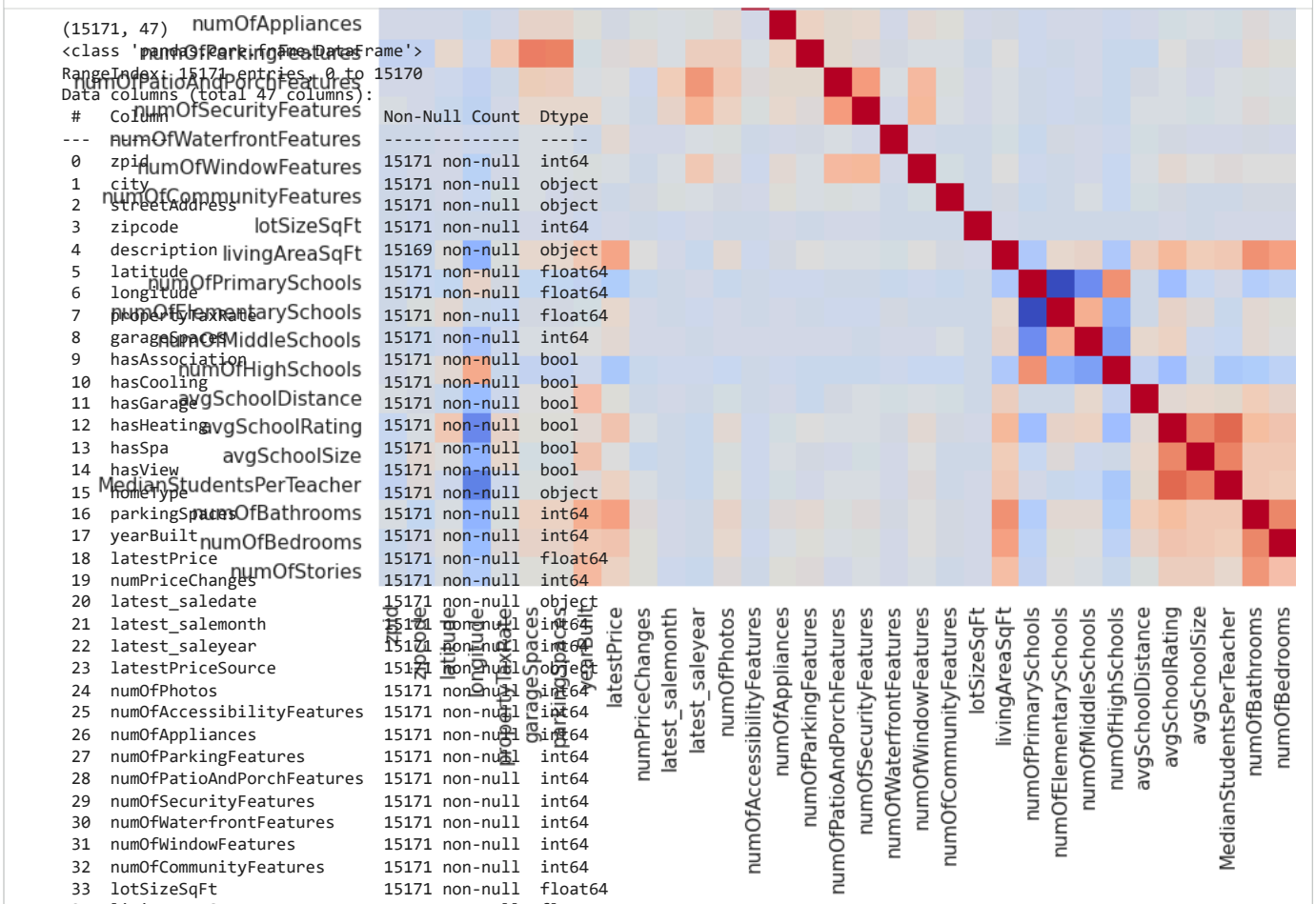

```
# CORRELATION ANALYSIS

plt.figure(figsize=(12, 10))
```

```
# numeric columns for correlation calculation
corr_matrix = df.select_dtypes(include=[np.number]).corr()
sns.heatmap(corr_matrix, annot=False, cmap='coolwarm')
plt.title("Correlation Heatmap")
plt.show()

target = "latestPrice"
if target in df.columns:
    print("\nTop correlations with target:\n")
    print(corr_matrix[target].sort_values(ascending=False))
```




```
print(df.shape)    # Number of rows & columns
print(df.info())   # Data types & non-null values
print(df.describe())   # Statistical summary
```

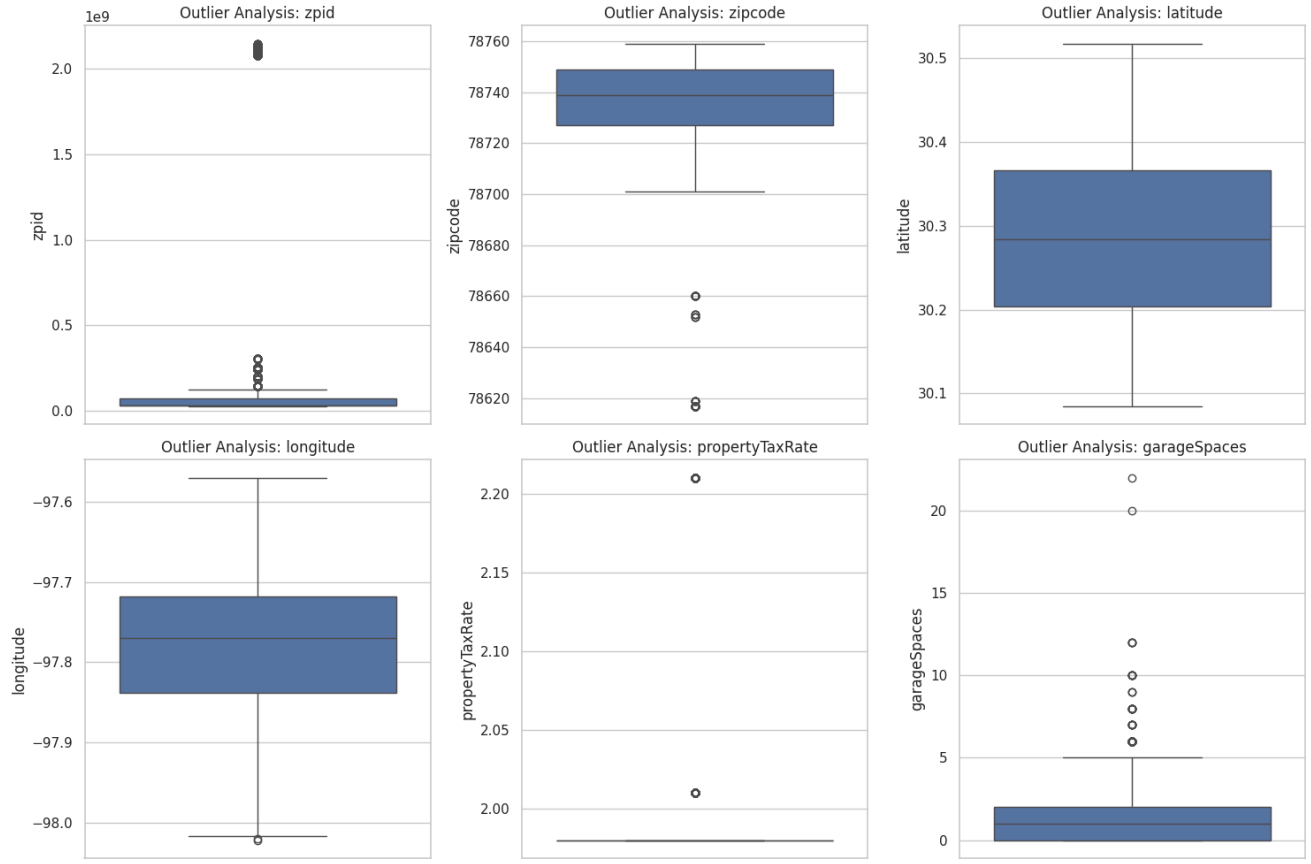
```
(15171, 47)
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 15171 entries, 0 to 15170
Data columns (total 47 columns):
 #   Column                      Non-Null Count  Dtype
---  ------                      --------------  -----
 0   zpid                        15171 non-null  int64
 1   city                        15171 non-null  object
 2   streetAddress               15171 non-null  object
 3   zipcode                     15171 non-null  int64
 4   description                 15169 non-null  object
 5   latitude                    15171 non-null  float64
 6   longitude                   15171 non-null  float64
 7   propertyTaxRate             15171 non-null  float64
 8   garageSpaces                15171 non-null  int64
 9   hasAssociation              15171 non-null  bool
 10  hasCooling                  15171 non-null  bool
 11  hasGarage                   15171 non-null  bool
 12  hasHeating                  15171 non-null  bool
 13  hasSpa                      15171 non-null  bool
 14  hasView                     15171 non-null  bool
 15  homeType                    15171 non-null  object
 16  parkingSpaces               15171 non-null  int64
 17  yearBuilt                   15171 non-null  int64
 18  latestPrice                 15171 non-null  float64
 19  numPriceChanges             15171 non-null  int64
 20  latest_saledate             15171 non-null  object
 21  latest_salemonth            15171 non-null  int64
 22  latest_saleyear             15171 non-null  int64
 23  latestPriceSource           15171 non-null  object
 24  numPhotos                   15171 non-null  int64
 25  numOfAccessibilityFeatures  15171 non-null  int64
 26  numOfAppliances             15171 non-null  int64
 27  numOfParkingFeatures        15171 non-null  int64
 28  numOfPatioAndPorchFeatures  15171 non-null  int64
 29  numOfSecurityFeatures       15171 non-null  int64
 30  numOfWaterfrontFeatures     15171 non-null  int64
 31  numOfWindowFeatures         15171 non-null  int64
 32  numOfCommunityFeatures      15171 non-null  int64
 33  lotSizeSqFt                 15171 non-null  float64
 34  livingAreaSqFt              15171 non-null  float64
 35  numOfPrimarySchools         15171 non-null  int64
 36  numOfElementarySchools      15171 non-null  int64
 37  numOfMiddleSchools          15171 non-null  int64
 38  numOfHighSchools            15171 non-null  int64
 39  avgSchoolDistance           15171 non-null  float64
 40  avgSchoolRating             15171 non-null  float64
 41  avgSchoolSize               15171 non-null  int64
 42  MedianStudentsPerTeacher    15171 non-null  float64
 43  numOfBathrooms              15171 non-null  int64
 44  numOfBedrooms               15171 non-null  int64
 45  numOfStories                15171 non-null  int64
 46  homeImage                   15171 non-null  object
dtypes: bool(6), float64(9), int64(25), object(7)
memory usage: 4.8+ MB
None
               zpid       zipcode      latitude     longitude  \
count  1.517100e+04  15171.000000  15171.000000  15171.000000
```



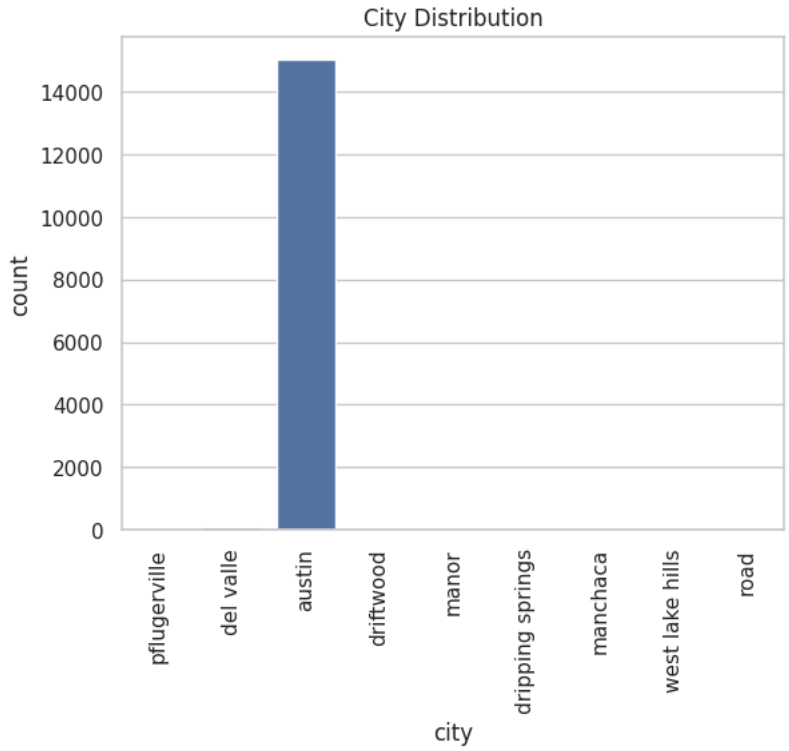

```
Top correlations with target:
latestPrice                   1.000000
numOfBathrooms                0.504738
livingAreaSqFt                0.467034
numOfBedrooms                 0.299839
avgSchoolRating               0.293101
numOfStories                  0.200555
MedianStudentsPerTeacher      0.198208
garageSpaces                  0.157443
parkingSpaces                 0.150811
numOfElementarySchools        0.139988
numPhotos                     0.134359
numOfMiddleSchools            0.127838
numOfWaterfrontFeatures       0.123979
numOfParkingFeatures          0.098384
avgSchoolDistance             0.094156
numOfPatioAndPorchFeatures    0.087936
```

```
# OUTLIER DETECTION USING BOXPLOTS

plt.figure(figsize=(15, 10))
for i, col in enumerate(numeric_cols[:6]):  # First 6 numerical features
```

```
    plt.subplot(2, 3, i+1)
    sns.boxplot(data=df, y=col)
    plt.title(f"Outlier Analysis: {col}")
plt.tight_layout()
plt.show()
```

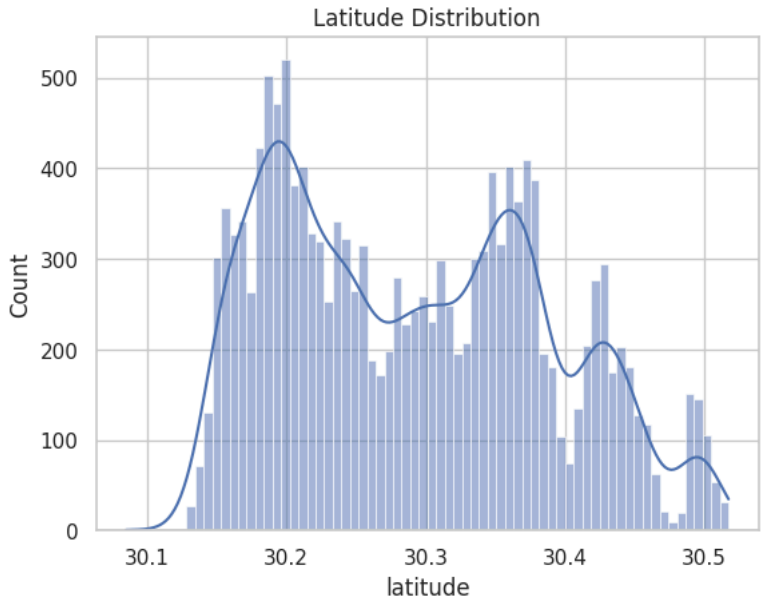



```
sns.countplot(x='city', data=df)
plt.xticks(rotation=90)
plt.title("City Distribution")
plt.show()
```

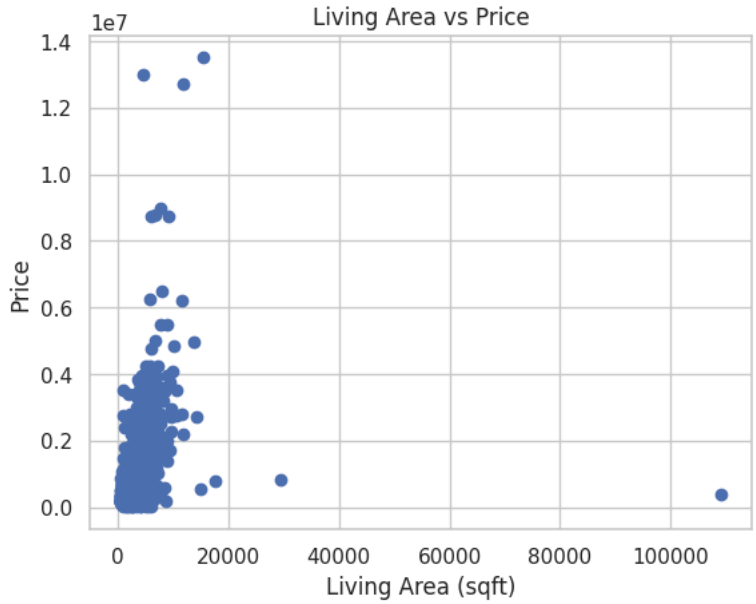



```
sns.histplot(df['latestPrice'], bins=70, kde=True)
plt.title("House Price Distribution")
plt.show()
```

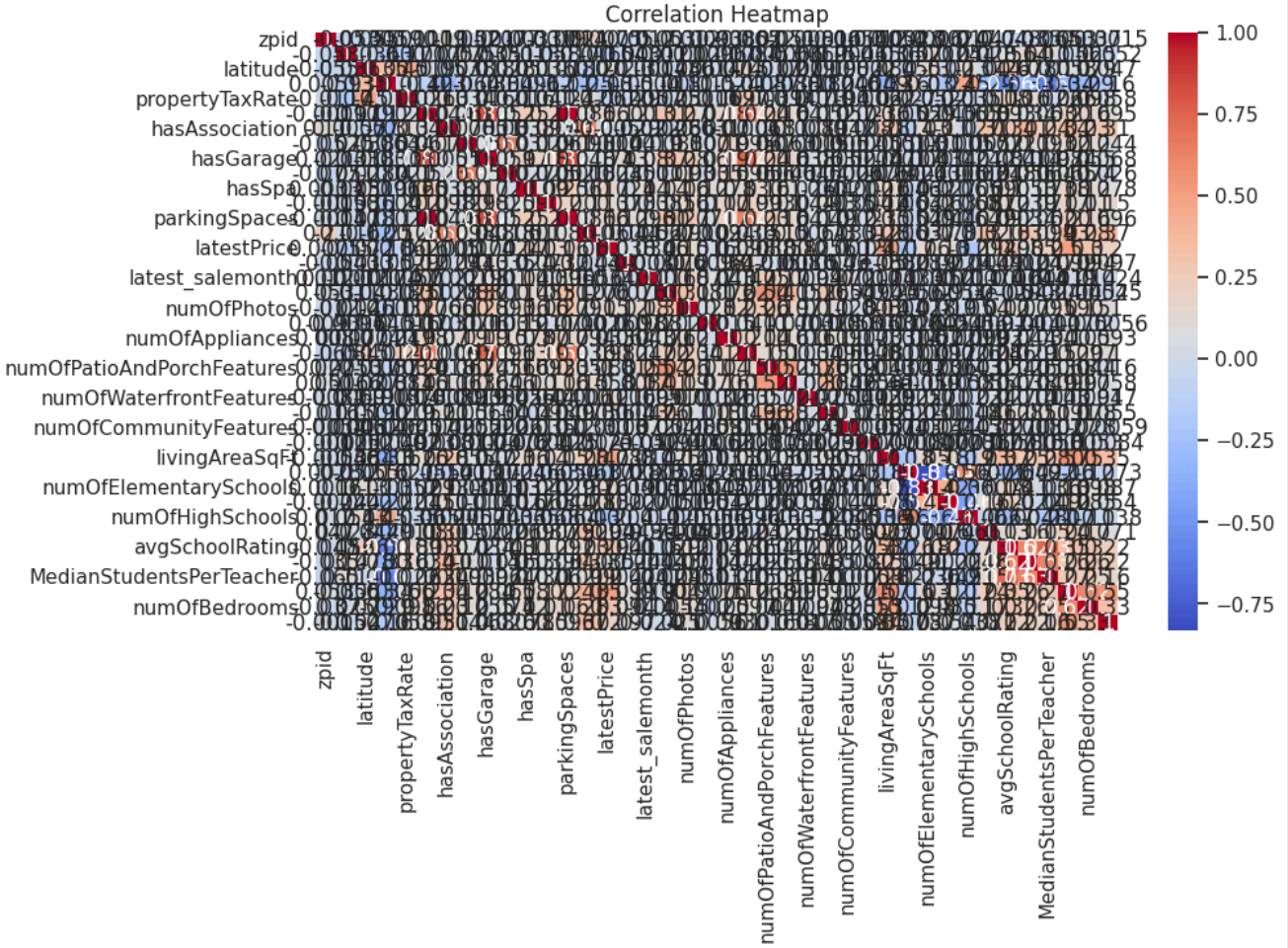

```
sns.histplot(df['latitude'], bins=70, kde=True)
plt.title("Latitude Distribution")
plt.show()
```




```
plt.scatter(df['livingAreaSqFt'], df['latestPrice'])
plt.xlabel("Living Area (sqft)")
plt.ylabel("Price")
plt.title("Living Area vs Price")
plt.show()
```

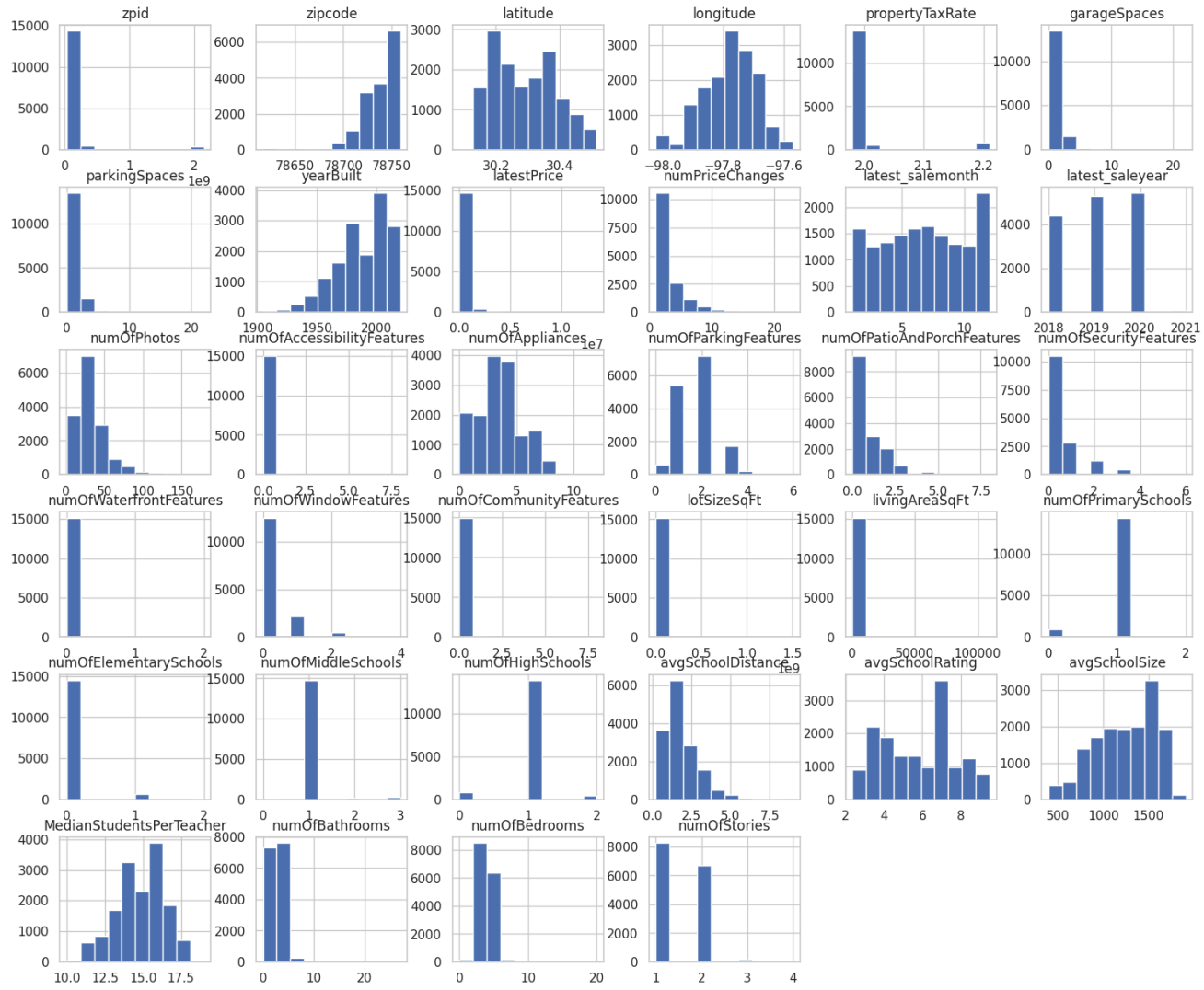



```
plt.figure(figsize=(10,6))
sns.heatmap(df.corr(numeric_only=True), annot=True, cmap='coolwarm')
plt.title("Correlation Heatmap")
plt.show()
```




```
df.hist(figsize=(18, 15))
plt.suptitle("Feature Distributions", fontsize=16)
plt.show()
```

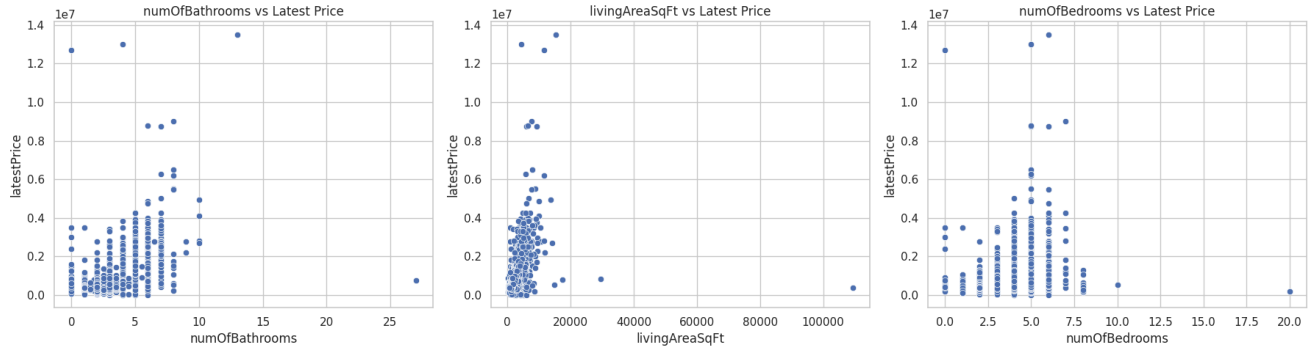
Feature Distributions



Start coding or generate with AI.

```
#  SCATTER PLOTS WITH TOP FEATURES


target = "latestPrice"

# Calculate the correlation matrix
corr = df.corr(numeric_only=True)

# Get top 3 correlated features with price
top_corr = corr[target].sort_values(ascending=False)[1:4]
print("Top correlated features:\n", top_corr)

plt.figure(figsize=(18,5))
for i, col in enumerate(top_corr.index):
    plt.subplot(1, 3, i+1)
    sns.scatterplot(x=df[col], y=df[target])
    plt.title(f"{col} vs Latest Price")

plt.tight_layout()
plt.show()
```

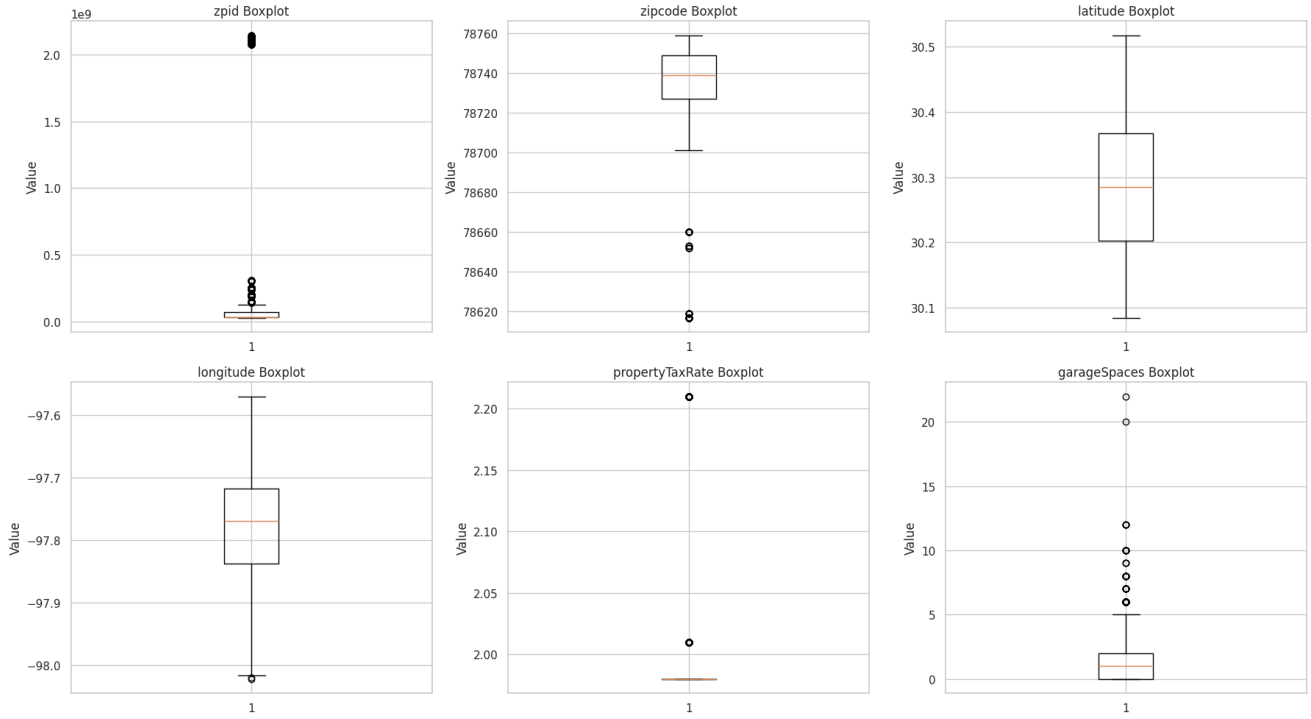
```
Top correlated features:
 numOfBathrooms    0.504738
livingAreaSqFt    0.467034
numOfBedrooms     0.299839
Name: latestPrice, dtype: float64
```




```
from scipy import stats

def detect_outliers_zscore(data, threshold=3):
    """Detect outliers using Z-score method"""
    z_scores = np.abs(stats.zscore(data))
    return np.where(z_scores > threshold)

# Define numerical_cols
numerical_cols = df.select_dtypes(include=[np.number]).columns

# Check outliers in numerical columns
if len(numerical_cols) > 0:
    outlier_counts = {}
    for col in numerical_cols[:10]:  # Check first 10 columns
        outliers = detect_outliers_zscore(df[col].dropna().values)
        outlier_counts[col] = len(outliers[0])

    outlier_df = pd.DataFrame.from_dict(outlier_counts, orient='index', columns=['Outlier Count'])
    outlier_df['Percentage'] = (outlier_df['Outlier Count'] / len(df)) * 100
    print("Outlier detection (Z-score > 3):")
    print(outlier_df.sort_values('Outlier Count', ascending=False))
```

```
# Visualize outliers with boxplots
fig, axes = plt.subplots(2, 3, figsize=(18, 10))
axes = axes.ravel()

for idx, col in enumerate(numerical_cols[:6]):
    axes[idx].boxplot(df[col].dropna())
    axes[idx].set_title(f'{col} Boxplot')
    axes[idx].set_ylabel('Value')

plt.tight_layout()
plt.show()
```

```
Outlier detection (Z-score > 3):
                 Outlier Count  Percentage
propertyTaxRate            856    5.642344
zpid                       371    2.445455
numPriceChanges            302    1.990640
latestPrice                239    1.575374
parkingSpaces              146    0.962362
garageSpaces               146    0.962362
zipcode                    142    0.935996
yearBuilt                   69    0.454815
latitude                     0    0.000000
longitude                    0    0.000000
```



```
# Categorical Analysis
print("\n CATEGORICAL FEATURE ANALYSIS")
print("-" * 40)

categorical_cols = df.select_dtypes(include=['object']).columns.tolist()
if categorical_cols:
    print(f"Categorical features: {categorical_cols}")

    # Plot categorical distributions
    n_cats = min(6, len(categorical_cols))
    fig, axes = plt.subplots(2, 3, figsize=(18, 10))
    axes = axes.ravel()

    for idx, col in enumerate(categorical_cols[:n_cats]):
        top_cats = df[col].value_counts().head(10)
        axes[idx].bar(range(len(top_cats)), top_cats.values)
        axes[idx].set_title(f'Top 10 {col}')
        axes[idx].set_xlabel(col)
        axes[idx].set_ylabel('Count')
        axes[idx].set_xticks(range(len(top_cats)))
        axes[idx].set_xticklabels(top_cats.index, rotation=45, ha='right')

    # Hide unused subplots
    for idx in range(n_cats, len(axes)):
        fig.delaxes(axes[idx])

    plt.tight_layout()
    plt.show()
```

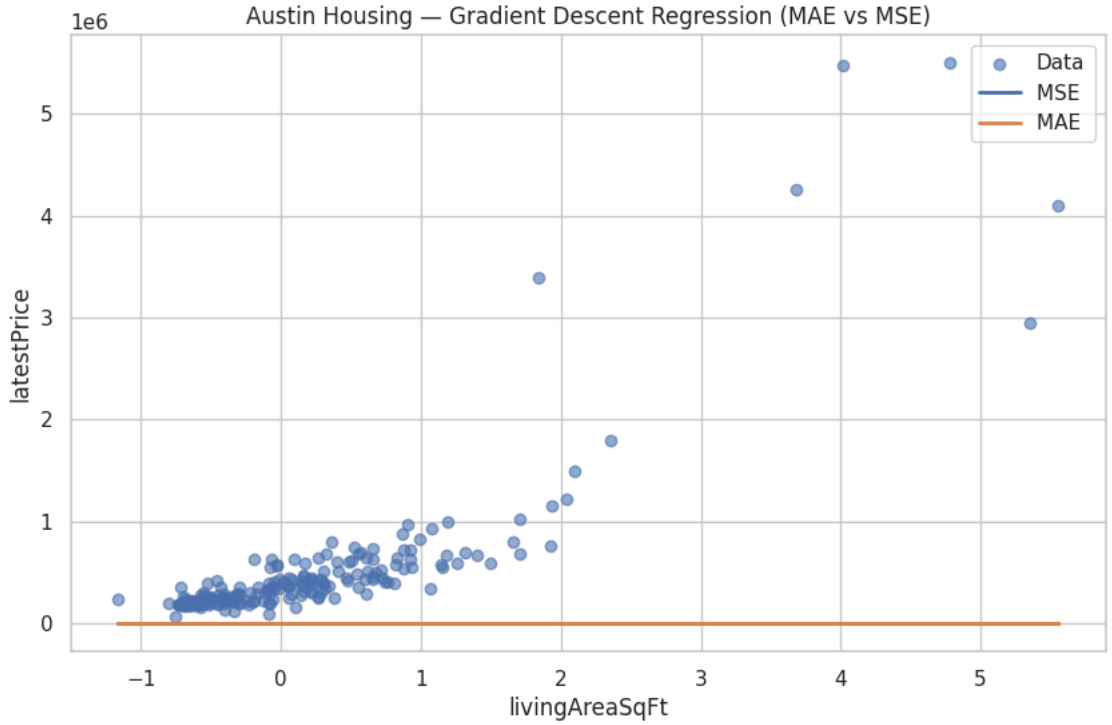
Show hidden output

Task 2. Hypothesis Formulation, Cost Function & Gradient Descent

```
# HYPOTHESIS FORMULATION

X = df[['livingAreaSqFt']].values
y = df['latestPrice'].values

# Normalize for faster convergence
X = (X - X.mean()) / X.std()

X = X.reshape(-1, 1)
y = y.reshape(-1, 1)

X_subset = X[:200].reshape(-1)    # Flatten for formulas
Y_subset = y[:200].reshape(-1)

m = len(Y_subset)

feature = "livingAreaSqFt"
target = "latestPrice"

def predict(X_val, theta0, theta1):
    return theta0 + theta1 * X_val


def gradient_descent_mae(X_val, Y_val, lr=0.0000001, iterations=50):
    theta0, theta1 = 0.0, 0.0

    for it in range(iterations):
        preds = predict(X_val, theta0, theta1)
        error = preds - Y_val

        sign = np.sign(error)

        grad0 = (1/m) * np.sum(sign)
        grad1 = (1/m) * np.sum(sign * X_val)

        theta0 -= lr * grad0
        theta1 -= lr * grad1

    return theta0, theta1
```

```
def gradient_descent_mse(X_val, Y_val, lr=0.0000000001, iterations=50):
    theta0, theta1 = 0.0, 0.0

    for it in range(iterations):
        preds = predict(X_val, theta0, theta1)
        error = preds - Y_val

        grad0 = (1/m) * np.sum(error)
        grad1 = (1/m) * np.sum(error * X_val)

        theta0 -= lr * grad0
        theta1 -= lr * grad1

    return theta0, theta1


theta0_mae, theta1_mae = gradient_descent_mae(X_subset, Y_subset, iterations=50)
theta0_mse, theta1_mse = gradient_descent_mse(X_subset, Y_subset, iterations=50)

print("Results after 50 iterations")
print("MAE -> Q0 =", round(theta0_mae, 4), ", Q1 =", round(theta1_mae, 4))
print("MSE -> Q0 =", round(theta0_mse, 4), ", Q1 =", round(theta1_mse, 4))



# VISUALIZATION
plt.figure(figsize=(10,6))
plt.scatter(X_subset, Y_subset, label="Data", alpha=0.6)

# MSE
plt.plot(
    X_subset,
    predict(X_subset, theta0_mse, theta1_mse),
    label="MSE ",
    linewidth=2
)

# MAE
plt.plot(
    X_subset,
    predict(X_subset, theta0_mae, theta1_mae),
    label="MAE ",
    linewidth=2
)

plt.legend()
plt.xlabel(feature)
plt.ylabel(target)
plt.title("Austin Housing – Gradient Descent Regression (MAE vs MSE)")
plt.show()
```

```
Results after 50 iterations
MAE -> Q0 = 0.0 , Q1 = 0.0
MSE -> Q0 = 0.0026 , Q1 = 0.0035
```

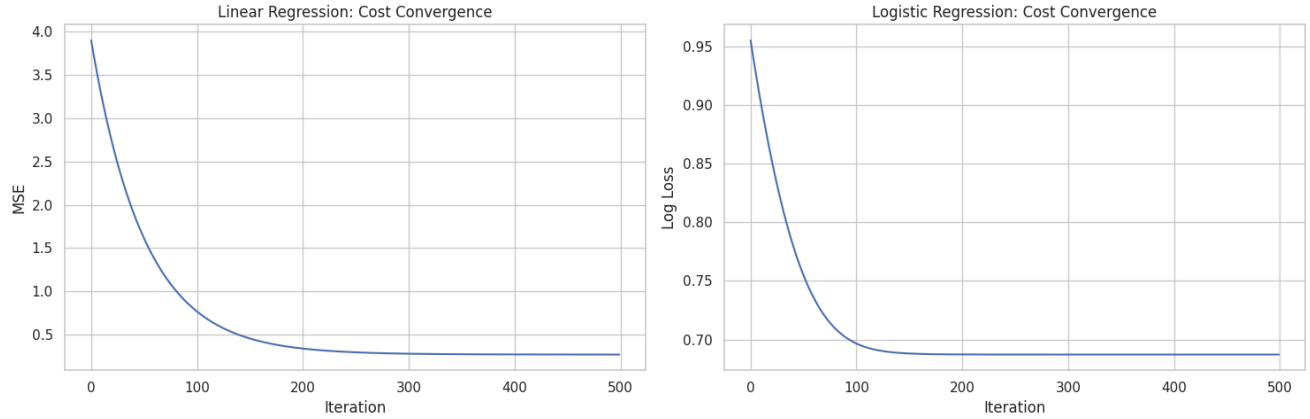



```
print("\nPREPARING DATA FOR FROM-SCRATCH IMPLEMENTATIONS")

# Pick 3 numerical features from Austin housing dataset
numerical_cols = df.select_dtypes(include=[np.number]).columns.tolist()
target_col = "price" if "price" in df.columns else numerical_cols[-1]

demo_cols = numerical_cols[:3]
X_demo = df[demo_cols].fillna(df[demo_cols].median())
y_demo = df[target_col].values

from sklearn.preprocessing import StandardScaler
scaler = StandardScaler()
X_demo_scaled = scaler.fit_transform(X_demo)

X_demo_bias = np.c_[np.ones((X_demo_scaled.shape[0], 1)), X_demo_scaled]

print(f"Using features: {demo_cols}")
print(f"X shape: {X_demo_bias.shape}")
print(f"y shape: {y_demo.shape}")


print("\n COST FUNCTION IMPLEMENTATION (FROM SCRATCH)")

def mean_squared_error_scratch(y_true, y_pred):
    return np.mean((y_true - y_pred) ** 2)

def logistic_cost_scratch(y_true, y_pred):
    epsilon = 1e-15
    y_pred = np.clip(y_pred, epsilon, 1 - epsilon)
    return -np.mean(y_true * np.log(y_pred) + (1 - y_true) * np.log(1 - y_pred))

def linear_hypothesis(X, theta):
    return X.dot(theta)

def sigmoid(z):
    return 1 / (1 + np.exp(-z))

def logistic_hypothesis(X, theta):
    return sigmoid(X.dot(theta))

# TEST COST FUNCTIONS
print("Testing cost functions...")
test_y_true = np.array([0, 1, 0, 1])
```

```
test_y_pred = np.array([0.1, 0.9, 0.2, 0.8])

print("MSE:", mean_squared_error_scratch(test_y_true, test_y_pred))
print("Log Loss:", logistic_cost_scratch(test_y_true, test_y_pred))


print("\n GRADIENT DESCENT IMPLEMENTATION (FROM SCRATCH)")

def gradient_descent_linear(X, y, theta, alpha, iterations):
    m = len(y)
    cost_history = []

    for i in range(iterations):
        predictions = X.dot(theta)
        errors = predictions - y

        gradient = (1/m) * X.T.dot(errors)
        theta = theta - alpha * gradient

        cost = mean_squared_error_scratch(y, predictions)
        cost_history.append(cost)

        if i % 100 == 0:
            print(f"Iteration {i}: Cost = {cost:.6f}")

    return theta, cost_history


def gradient_descent_logistic(X, y, theta, alpha, iterations):
    m = len(y)
    cost_history = []

    for i in range(iterations):
        predictions = logistic_hypothesis(X, theta)
        gradient = (1/m) * X.T.dot(predictions - y)

        theta = theta - alpha * gradient

        cost = logistic_cost_scratch(y, predictions)
        cost_history.append(cost)

        if i % 100 == 0:
            print(f"Iteration {i}: Cost = {cost:.6f}")

    return theta, cost_history

print("\nRunning Gradient Descent for Linear Regression...")
np.random.seed(42)
theta_init = np.random.randn(X_demo_bias.shape[1])
theta_linear, cost_history_linear = gradient_descent_linear(
    X_demo_bias, y_demo, theta_init, alpha=0.01, iterations=500
)

print("\nRunning Gradient Descent for Logistic Regression...")
theta_init = np.random.randn(X_demo_bias.shape[1])

y_binary = (y_demo > y_demo.mean()).astype(int)

theta_logistic, cost_history_logistic = gradient_descent_logistic(
    X_demo_bias, y_binary, theta_init, alpha=0.1, iterations=500
)

fig, axes = plt.subplots(1, 2, figsize=(15, 5))

axes[0].plot(cost_history_linear)
axes[0].set_title('Linear Regression: Cost Convergence')
axes[0].set_xlabel('Iteration')
axes[0].set_ylabel('MSE')
axes[0].grid(True)

axes[1].plot(cost_history_logistic)
axes[1].set_title('Logistic Regression: Cost Convergence')
axes[1].set_xlabel('Iteration')
axes[1].set_ylabel('Log Loss')
axes[1].grid(True)

plt.tight_layout()
```

```
plt.show()

print("\nFinal Linear Regression Parameters:", theta_linear)
print("Final Logistic Regression Parameters:", theta_logistic)
```

```
PREPARING DATA FOR FROM-SCRATCH IMPLEMENTATIONS
Using features: ['zpid', 'zipcode', 'latitude']
X shape: (15171, 4)
y shape: (15171,)

 COST FUNCTION IMPLEMENTATION (FROM SCRATCH)
Testing cost functions...
MSE: 0.024999999999999994
Log Loss: 0.164252033486018

 GRADIENT DESCENT IMPLEMENTATION (FROM SCRATCH)

Running Gradient Descent for Linear Regression...
Iteration 0: Cost = 3.895021
Iteration 100: Cost = 0.769271
Iteration 200: Cost = 0.342202
Iteration 300: Cost = 0.282785
Iteration 400: Cost = 0.274365

Running Gradient Descent for Logistic Regression...
Iteration 0: Cost = 0.954995
Iteration 100: Cost = 0.696605
Iteration 200: Cost = 0.687291
Iteration 300: Cost = 0.687212
Iteration 400: Cost = 0.687211
```




```
Final Linear Regression Parameters: [ 1.46089605  0.00258089 -0.01966255  0.03487929]
Final Logistic Regression Parameters: [-0.18106871 -0.01817005 -0.07036817  0.09686465]
```

```
######################################## 3. Linear Regression
```

```
# LINEAR REGRESSION

print("TASK 3: LINEAR REGRESSION")

print("\n3.1 DATA PREPARATION FOR LINEAR REGRESSION")

target_col = 'latestPrice'
y_reg = df[target_col]

# Select features for regression
numerical_cols = df.select_dtypes(include=[np.number]).columns.tolist()
reg_features = [col for col in numerical_cols if col != target_col][:8]
X_reg = df[reg_features].fillna(df[reg_features].median())
```

```
print(f"Regression Features: {reg_features}")
print(f"X shape: {X_reg.shape}, y shape: {y_reg.shape}")


from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
from sklearn.linear_model import LinearRegression
from sklearn.metrics import mean_squared_error, mean_absolute_error, r2_score


X_train_reg, X_test_reg, y_train_reg, y_test_reg = train_test_split(
    X_reg, y_reg, test_size=0.2, random_state=42
)

scaler_reg = StandardScaler()
X_train_reg_scaled = scaler_reg.fit_transform(X_train_reg)
X_test_reg_scaled = scaler_reg.transform(X_test_reg)

# 3.2 Linear Regression from Scratch
print("\n3.2 LINEAR REGRESSION FROM SCRATCH")
print("-" * 40)

class LinearRegressionScratch:
    def __init__(self, learning_rate=0.01, iterations=1000):
        self.learning_rate = learning_rate
        self.iterations = iterations
        self.theta = None
        self.cost_history = []

    def fit(self, X, y):
        # Add bias term
        X_b = np.c_[np.ones((X.shape[0], 1)), X]

        # Initialize parameters
        m, n = X_b.shape
        self.theta = np.random.randn(n, 1)
        y = y.reshape(-1, 1)

        # Gradient descent
        for i in range(self.iterations):
            predictions = X_b.dot(self.theta)

            errors = predictions - y

            gradients = (2/m) * X_b.T.dot(errors)

            # Update parameters
            self.theta = self.theta - self.learning_rate * gradients

            # Calculate cost
            cost = np.mean(errors ** 2)
            self.cost_history.append(cost)

            if i % 100 == 0:
                print(f"Iteration {i}: Cost = {cost:.6f}")

        return self

    def predict(self, X):
        X_b = np.c_[np.ones((X.shape[0], 1)), X]
        return X_b.dot(self.theta).flatten()

    def score(self, X, y):
        y_pred = self.predict(X)
        ss_res = np.sum((y - y_pred) ** 2)
        ss_tot = np.sum((y - np.mean(y)) ** 2)
        return 1 - (ss_res / ss_tot)


print("Training Linear Regression from scratch...")
lr_scratch = LinearRegressionScratch(learning_rate=0.01, iterations=1000)
lr_scratch.fit(X_train_reg_scaled, y_train_reg.values)

y_pred_scratch = lr_scratch.predict(X_test_reg_scaled)

mse_scratch = mean_squared_error(y_test_reg, y_pred_scratch)
mae_scratch = mean_absolute_error(y_test_reg, y_pred_scratch)
```

```
r2_scratch = r2_score(y_test_reg, y_pred_scratch)

print(f"\nFrom-Scratch Model Performance:")
print(f"MSE: {mse_scratch:.4f}")
print(f"MAE: {mae_scratch:.4f}")
print(f"R² Score: {r2_scratch:.4f}")


print("\n3.3 LINEAR REGRESSION USING LIBRARY (scikit-learn)")

# Train
lr_library = LinearRegression()
lr_library.fit(X_train_reg_scaled, y_train_reg)

y_pred_library = lr_library.predict(X_test_reg_scaled)

mse_library = mean_squared_error(y_test_reg, y_pred_library)
mae_library = mean_absolute_error(y_test_reg, y_pred_library)
r2_library = r2_score(y_test_reg, y_pred_library)

print("Library Model Performance:")
print(f"MSE: {mse_library:.4f}")
print(f"MAE: {mae_library:.4f}")
print(f"R² Score: {r2_library:.4f}")


print("\n3.4 COEFFICIENT COMPARISON")

# Get coefficients
coeff_scratch = lr_scratch.theta.flatten()[1:]  # Exclude bias
coeff_library = lr_library.coef_

coeff_df = pd.DataFrame({
    'Feature': reg_features,
    'From_Scratch': coeff_scratch[:len(reg_features)],
    'Library': coeff_library
})

print("Coefficient Comparison:")
print(coeff_df)

# 3.5 Visualization
fig, axes = plt.subplots(2, 3, figsize=(18, 10))

# Cost convergence
axes[0, 0].plot(lr_scratch.cost_history)
axes[0, 0].set_title('From-Scratch: Cost Convergence')
axes[0, 0].set_xlabel('Iteration')
axes[0, 0].set_ylabel('MSE')
axes[0, 0].grid(True)

# Actual vs Predicted (Scratch)
axes[0, 1].scatter(y_test_reg, y_pred_scratch, alpha=0.5)
axes[0, 1].plot([y_test_reg.min(), y_test_reg.max()],
                [y_test_reg.min(), y_test_reg.max()], 'r--', lw=2)
axes[0, 1].set_title('From-Scratch: Actual vs Predicted')
axes[0, 1].set_xlabel('Actual')
axes[0, 1].set_ylabel('Predicted')

# Actual vs Predicted (Library)
axes[0, 2].scatter(y_test_reg, y_pred_library, alpha=0.5)
axes[0, 2].plot([y_test_reg.min(), y_test_reg.max()],
                [y_test_reg.min(), y_test_reg.max()], 'r--', lw=2)
axes[0, 2].set_title('Library: Actual vs Predicted')
axes[0, 2].set_xlabel('Actual')
axes[0, 2].set_ylabel('Predicted')

# Residuals (Scratch)
axes[1, 0].scatter(y_pred_scratch, y_test_reg - y_pred_scratch, alpha=0.5)
axes[1, 0].axhline(y=0, color='r', linestyle='--')
axes[1, 0].set_title('From-Scratch: Residual Plot')
axes[1, 0].set_xlabel('Predicted')
axes[1, 0].set_ylabel('Residuals')

# Residuals (Library)
axes[1, 1].scatter(y_pred_library, y_test_reg - y_pred_library, alpha=0.5)
axes[1, 1].axhline(y=0, color='r', linestyle='--')
```

```
axes[1, 1].set_title('Library: Residual Plot')
axes[1, 1].set_xlabel('Predicted')
axes[1, 1].set_ylabel('Residuals')

# Coefficient Comparison
x_pos = np.arange(len(reg_features))
width = 0.35
axes[1, 2].bar(x_pos - width/2, coeff_scratch, width, label='From-Scratch')
axes[1, 2].bar(x_pos + width/2, coeff_library, width, label='Library')
axes[1, 2].set_title('Coefficient Comparison')
axes[1, 2].set_xlabel('Features')
axes[1, 2].set_ylabel('Coefficient Value')
axes[1, 2].set_xticks(x_pos)
axes[1, 2].set_xticklabels(reg_features, rotation=45, ha='right')
axes[1, 2].legend()

plt.tight_layout()
plt.show()
```

```
TASK 3: LINEAR REGRESSION

3.1 DATA PREPARATION FOR LINEAR REGRESSION
Regression Features: ['zpid', 'zipcode', 'latitude', 'longitude', 'propertyTaxRate', 'garageSpaces', 'parkingSpaces', 'yearBuilt
X shape: (15171, 8), y shape: (15171,)

3.2 LINEAR REGRESSION FROM SCRATCH
----------------------------------------
Training Linear Regression from scratch...
Iteration 0: Cost = 460500185091.336670
Iteration 100: Cost = 178856174791.739532
Iteration 200: Cost = 171368364928.012268
Iteration 300: Cost = 170700867552.814880
Iteration 400: Cost = 170579606292.686371
Iteration 500: Cost = 170555045598.828470
```

Start coding or generate with AI.

```
Iteration 800: Cost = 170548726406.433648
```

```
# TASK 4: LOGISTIC REGRESSION

print("TASK 4: LOGISTIC REGRESSION")
# Import necessary modules for classification metrics
from sklearn.metrics import (
    accuracy_score,
    precision_score,
    recall_score,
    f1_score,
    roc_auc_score,
    confusion_matrix,
    roc_curve,
    precision_recall_curve
)
from sklearn.linear_model import LogisticRegression

# 4.1 DATA PREPARATION FOR LOGISTIC REGRESSION
print("\n4.1 DATA PREPARATION FOR LOGISTIC REGRESSION")

target_col = 'latestPrice'
y_class = (df[target_col] > df[target_col].median()).astype(int)

numerical_cols = df.select_dtypes(include=[np.number]).columns.tolist()
class_features = [col for col in numerical_cols if col != target_col][:10]
X_class = df[class_features].fillna(df[class_features].median())

print(f"Classification Features: {class_features}")
print(f"Target distribution:\n{y_class.value_counts()}")

# Split data
X_train_class, X_test_class, y_train_class, y_test_class = train_test_split(
    X_class, y_class, test_size=0.2, random_state=42, stratify=y_class
)

# Scale features
scaler_class = StandardScaler()
X_train_class_scaled = scaler_class.fit_transform(X_train_class)
X_test_class_scaled = scaler_class.transform(X_test_class)

# 4.2 Logistic Regression from Scratch
print("\n4.2 LOGISTIC REGRESSION FROM SCRATCH")

class LogisticRegressionScratch:

    def __init__(self, learning_rate=0.1, iterations=1000, tol=1e-4):
        self.learning_rate = learning_rate
        self.iterations = iterations
        self.tol = tol
        self.theta = None
        self.cost_history = []

    def sigmoid(self, z):
        return 1 / (1 + np.exp(-z))

    def cost_function(self, X, y, theta):
        m = len(y)
        h = self.sigmoid(X.dot(theta))
        # Clip h to avoid log(0) errors
        h = np.clip(h, 1e-10, 1 - 1e-10)
```

```
            cost = (-1/m) * np.sum(y * np.log(h) + (1-y) * np.log(1-h))
            return cost

        def fit(self, X, y):
            # Add bias term
            X_b = np.c_[np.ones((X.shape[0], 1)), X]

            # Initialize parameters
            m, n = X_b.shape
            self.theta = np.random.randn(n, 1)
            y = y.values.reshape(-1, 1) if hasattr(y, 'values') else y.reshape(-1, 1)

            # Gradient descent
            for i in range(self.iterations):
                # Calculate predictions
                z = X_b.dot(self.theta)
                h = self.sigmoid(z)

                # Calculate gradient
                gradient = (1/m) * X_b.T.dot(h - y)

                # Update parameters
                self.theta = self.theta - self.learning_rate * gradient

                # Calculate cost
                cost = self.cost_function(X_b, y, self.theta)
                self.cost_history.append(cost)

                # Check convergence
                if i > 0 and abs(self.cost_history[-1] - self.cost_history[-2]) < self.tol:
                    print(f"Converged at iteration {i}")
                    break

                if i % 100 == 0:
                    print(f"Iteration {i}: Cost = {cost:.6f}")

            return self

        def predict_proba(self, X):
            """Predict probabilities"""
            X_b = np.c_[np.ones((X.shape[0], 1)), X]
            return self.sigmoid(X_b.dot(self.theta)).flatten()

        def predict(self, X, threshold=0.5):
            """Make binary predictions"""
            proba = self.predict_proba(X)
            return (proba >= threshold).astype(int)

        def score(self, X, y):
            """Calculate accuracy"""
            y_pred = self.predict(X)
            return accuracy_score(y, y_pred)

    # Train from-scratch model
    print("Training Logistic Regression from scratch...")
    logreg_scratch = LogisticRegressionScratch(learning_rate=0.1, iterations=1000)
    logreg_scratch.fit(X_train_class_scaled, y_train_class)

    # Make predictions
    y_pred_scratch_log = logreg_scratch.predict(X_test_class_scaled)
    y_prob_scratch_log = logreg_scratch.predict_proba(X_test_class_scaled)

    # Calculate metrics
    accuracy_scratch_log = accuracy_score(y_test_class, y_pred_scratch_log)
    precision_scratch_log = precision_score(y_test_class, y_pred_scratch_log)
    recall_scratch_log = recall_score(y_test_class, y_pred_scratch_log)
    f1_scratch_log = f1_score(y_test_class, y_pred_scratch_log)
    auc_scratch_log = roc_auc_score(y_test_class, y_prob_scratch_log)

    print(f"\nFrom-Scratch Model Performance:")
    print(f"Accuracy: {accuracy_scratch_log:.4f}")
    print(f"Precision: {precision_scratch_log:.4f}")
    print(f"Recall: {recall_scratch_log:.4f}")
    print(f"F1-Score: {f1_scratch_log:.4f}")
    print(f"ROC-AUC: {auc_scratch_log:.4f}")

    # 4.3 Logistic Regression using Library
```

```
print("\n4.3 LOGISTIC REGRESSION USING LIBRARY")
print("-" * 40)

# Train library model
logreg_library = LogisticRegression(max_iter=1000, random_state=42)
logreg_library.fit(X_train_class_scaled, y_train_class)

# Make predictions
y_pred_library_log = logreg_library.predict(X_test_class_scaled)
y_prob_library_log = logreg_library.predict_proba(X_test_class_scaled)[:, 1]

# Calculate metrics
accuracy_library_log = accuracy_score(y_test_class, y_pred_library_log)
precision_library_log = precision_score(y_test_class, y_pred_library_log)
recall_library_log = recall_score(y_test_class, y_pred_library_log)
f1_library_log = f1_score(y_test_class, y_pred_library_log)
auc_library_log = roc_auc_score(y_test_class, y_prob_library_log)

print("Library Model Performance:")
print(f"Accuracy: {accuracy_library_log:.4f}")
print(f"Precision: {precision_library_log:.4f}")
print(f"Recall: {recall_library_log:.4f}")
print(f"F1-Score: {f1_library_log:.4f}")
print(f"ROC-AUC: {auc_library_log:.4f}")

# 4.4 Threshold Analysis
print("\n4.4 THRESHOLD ANALYSIS AND PROBABILITY INTERPRETATION")
print("-" * 40)

# Analyze different thresholds
thresholds = np.arange(0.1, 1.0, 0.1)
results = []

for threshold in thresholds:
    y_pred_thresh = (y_prob_library_log >= threshold).astype(int)
    accuracy = accuracy_score(y_test_class, y_pred_thresh)
    precision = precision_score(y_test_class, y_pred_thresh, zero_division=0)
    recall = recall_score(y_test_class, y_pred_thresh)
    f1 = f1_score(y_test_class, y_pred_thresh)
    results.append({
        'Threshold': threshold,
        'Accuracy': accuracy,
        'Precision': precision,
        'Recall': recall,
        'F1': f1
    })

threshold_df = pd.DataFrame(results)
print("Performance at Different Thresholds:")
print(threshold_df)

# Find optimal threshold (max F1)
optimal_idx = threshold_df['F1'].idxmax()
optimal_threshold = threshold_df.loc[optimal_idx, 'Threshold']
print(f"\nOptimal Threshold (max F1): {optimal_threshold:.2f}")

# 4.5 Visualization
fig, axes = plt.subplots(2, 3, figsize=(18, 10))

# Cost convergence
axes[0, 0].plot(logreg_scratch.cost_history)
axes[0, 0].set_title('From-Scratch: Cost Convergence')
axes[0, 0].set_xlabel('Iteration')
axes[0, 0].set_ylabel('Log Loss')
axes[0, 0].grid(True)

# Confusion Matrix (Scratch)
cm_scratch = confusion_matrix(y_test_class, y_pred_scratch_log)
sns.heatmap(cm_scratch, annot=True, fmt='d', cmap='Blues', ax=axes[0, 1])
axes[0, 1].set_title('From-Scratch: Confusion Matrix')
axes[0, 1].set_xlabel('Predicted')
axes[0, 1].set_ylabel('Actual')

# Confusion Matrix (Library)
cm_library = confusion_matrix(y_test_class, y_pred_library_log)
sns.heatmap(cm_library, annot=True, fmt='d', cmap='Greens', ax=axes[0, 2])
axes[0, 2].set_title('Library: Confusion Matrix')
```

```
axes[0, 2].set_xlabel('Predicted')
axes[0, 2].set_ylabel('Actual')

# ROC Curve
fpr_scratch, tpr_scratch, _ = roc_curve(y_test_class, y_prob_scratch_log)
fpr_library, tpr_library, _ = roc_curve(y_test_class, y_prob_library_log)

axes[1, 0].plot(fpr_scratch, tpr_scratch, label=f'From-Scratch (AUC={auc_scratch_log:.3f})')
axes[1, 0].plot(fpr_library, tpr_library, label=f'Library (AUC={auc_library_log:.3f})')
axes[1, 0].plot([0, 1], [0, 1], 'k--', label='Random')
axes[1, 0].set_title('ROC Curves')
axes[1, 0].set_xlabel('False Positive Rate')
axes[1, 0].set_ylabel('True Positive Rate')
axes[1, 0].legend()
axes[1, 0].grid(True)

# Precision-Recall Curve
precision_scratch, recall_scratch, _ = precision_recall_curve(y_test_class, y_prob_scratch_log)
precision_library, recall_library, _ = precision_recall_curve(y_test_class, y_prob_library_log)

axes[1, 1].plot(recall_scratch, precision_scratch, label='From-Scratch')
axes[1, 1].plot(recall_library, precision_library, label='Library')
axes[1, 1].set_title('Precision-Recall Curves')
axes[1, 1].set_xlabel('Recall')
axes[1, 1].set_ylabel('Precision')
axes[1, 1].legend()
axes[1, 1].grid(True)

# Threshold Analysis
axes[1, 2].plot(threshold_df['Threshold'], threshold_df['Accuracy'], label='Accuracy')
axes[1, 2].plot(threshold_df['Threshold'], threshold_df['Precision'], label='Precision')
axes[1, 2].plot(threshold_df['Threshold'], threshold_df['Recall'], label='Recall')
axes[1, 2].plot(threshold_df['Threshold'], threshold_df['F1'], label='F1', linewidth=3)
axes[1, 2].axvline(x=optimal_threshold, color='r', linestyle='--', label=f'Optimal ({optimal_threshold:.2f})')
axes[1, 2].set_title('Threshold Analysis')
axes[1, 2].set_xlabel('Threshold')
axes[1, 2].set_ylabel('Score')
axes[1, 2].legend()
axes[1, 2].grid(True)

plt.tight_layout()
plt.show()
```

```
TASK 4: LOGISTIC REGRESSION

4.1 DATA PREPARATION FOR LOGISTIC REGRESSION
Classification Features: ['zpid', 'zipcode', 'latitude', 'longitude', 'propertyTaxRate', 'garageSpaces', 'parkingSpaces', 'yearB
Target distribution:
latestPrice
0    7621
1    7550
Name: count, dtype: int64

4.2 LOGISTIC REGRESSION FROM SCRATCH
Training Logistic Regression from scratch...
Iteration 0: Cost = 1.368806
Iteration 100: Cost = 0.569261
Converged at iteration 156

From-Scratch Model Performance:
Accuracy: 0.7209
Precision: 0.7285
Recall: 0.7009
```

```
##or
```

```
from sklearn.metrics import confusion_matrix
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import accuracy_score, precision_score, recall_score, f1_score
from sklearn.model_selection import train_test_split


target = "latestPrice"

# Select numeric features
num_features = df.select_dtypes(include=[np.number]).columns.tolist()
num_features.remove(target)

# Create binary target
median_price = df[target].median()
df["y"] = (df[target] > median_price).astype(int)

# Prepare data
X = df[num_features].values
y = df["y"].values

# Train-test split
x_train, x_test, y_train, y_test = train_test_split(
    X, y, test_size=0.25, random_state=42
)

# Model
model = LogisticRegression(max_iter=2000)
model.fit(x_train, y_train)

# Prediction
y_pred = model.predict(x_test)

print("Intercept:", model.intercept_)
print("Coefficients:", model.coef_)
print("Confusion matrix :", confusion_matrix(y_test, y_pred))

accuracy  = accuracy_score(y_test, y_pred)
precision = precision_score(y_test, y_pred)
recall    = recall_score(y_test, y_pred)
f1        = f1_score(y_test, y_pred)
cm        = confusion_matrix(y_test, y_pred)

print(f"Predictions: {y_pred}")
print(f"Accuracy: {accuracy:.4f}")
print(f"Precision: {precision:.4f}")
print(f"Recall: {recall:.4f}")
print(f"F1-Score: {f1:.4f}")
print(f"Confusion Matrix:\n{cm}")

print(f"/n")

from sklearn.model_selection import train_test_split
from sklearn.metrics import confusion_matrix, accuracy_score, precision_score, recall_score, f1_score

# Define target explicitly for robustness
```

```
target = "latestPrice"

# Select numeric features
num_features = df.select_dtypes(include=[np.number]).columns.tolist()
num_features.remove(target)

# Create binary classification target
median_price = df[target].median()
df["y"] = (df[target] > median_price).astype(int)

# Prepare X and y
X = df[num_features].values
y = df["y"].values.reshape(-1,1)

# Train-test split
x_train, x_test, y_train, y_test = train_test_split(
    X, y, test_size=0.25, random_state=42
)

m, n = x_train.shape

# Standardization for stability
mean = x_train.mean(axis=0)
std  = x_train.std(axis=0)
x_train = (x_train - mean) / std
x_test  = (x_test - mean) / std

# Add bias term
x_train_b = np.hstack([np.ones((m,1)), x_train])
x_test_b  = np.hstack([np.ones((x_test.shape[0],1)), x_test])


def sigmoid(z):
    return 1 / (1 + np.exp(-z))


def compute_cost(X, y, theta):
    m = len(y)
    h = sigmoid(X.dot(theta))
    cost = -(1/m) * np.sum(y*np.log(h+1e-9) + (1-y)*np.log(1-h+1e-9))
    return cost


def gradient_descent(X, y, learning_rate=0.01, iterations=2000):
    m, n = X.shape
    theta = np.zeros((n,1))
    cost_history = []

    for i in range(iterations):
        h = sigmoid(X.dot(theta))
        gradient = (1/m) * X.T.dot(h - y)
        theta = theta - learning_rate * gradient
        cost_history.append(compute_cost(X, y, theta))

    return theta, cost_history

# Train model
theta, cost_hist = gradient_descent(x_train_b, y_train)


def predict(X, theta):
    prob = sigmoid(X.dot(theta))
    return (prob >= 0.5).astype(int)

# Predict on test set
y_pred = predict(x_test_b, theta)


accuracy  = accuracy_score(y_test, y_pred)
precision = precision_score(y_test, y_pred)
recall    = recall_score(y_test, y_pred)
f1        = f1_score(y_test, y_pred)
cm        = confusion_matrix(y_test, y_pred)

print("Theta shape:", theta.shape)
print("First 10 coefficients:", theta[:10].flatten())
print("Final Cost:", cost_hist[-1])
```

```
print("Confusion matrix :", cm)
print(f"Predictions: {y_pred.flatten()[:15]}")
print(f"Accuracy: {accuracy:.4f}")
print(f"Precision: {precision:.4f}")
print(f"Recall: {recall:.4f}")
print(f"F1-Score: {f1:.4f}")
print(f"Confusion Matrix:\n{cm}")
```

```
Intercept: [2.15590517e-07]
Coefficients: [[ 7.41419095e-10  8.07884520e-04  4.45781708e-05 -7.46267619e-05
  -4.09475382e-05  6.03023389e-04  5.97610634e-04 -3.53432921e-02
  -1.19132691e-03  1.03169718e-03  1.11014246e-03  1.29781152e-02
   5.51475738e-05  7.80064368e-05  3.22690530e-04  7.57923045e-04
   3.15358858e-04  7.33439228e-06  2.12259711e-04 -1.31163089e-05
   3.20149001e-08  1.99031283e-03 -5.45268354e-05  1.64179332e-05
   4.57205760e-05 -2.51671261e-04 -7.47409491e-04  3.37757924e-03
  -2.29806110e-04  2.69079035e-03  3.44496175e-05 -3.07256513e-04
  -2.17648383e-04]]
Confusion matrix : [[1525  385]
 [ 512 1371]]
Predictions: [0 1 0 ... 0 0 0]
Accuracy: 0.7635
Precision: 0.7808
Recall: 0.7281
F1-Score: 0.7535
Confusion Matrix:
[[1525  385]
 [ 512 1371]]
/n
Theta shape: (35, 1)
First 10 coefficients: [ 0.0190141   0.07666764 -0.24285314  0.12959888 -0.10865268 -0.20792025
  0.0324544   0.03020591 -0.24862457 -0.1216749 ]
Final Cost: 0.053431044196736506
Confusion matrix : [[1910    0]
 [   0 1883]]
Predictions: [0 1 0 0 1 0 1 0 0 0 0 0 1 0 0]
Accuracy: 1.0000
Precision: 1.0000
Recall: 1.0000
F1-Score: 1.0000
Confusion Matrix:
[[1910    0]
 [   0 1883]]
```

```
##############task 5. Feature Regularization to Prevent Overfitting
```

```
from sklearn.linear_model import Ridge, Lasso
from sklearn.metrics import r2_score, mean_squared_error

# Define target explicitly for robustness
target = "latestPrice"

# Use numeric features only
num_features = df.select_dtypes(include=[np.number]).columns.tolist()
num_features.remove(target)

x = df[num_features]
y = df[target]


def regression_model(x, y, alpha):
    Ridge_model = Ridge(alpha)
    Ridge_model.fit(x, y)
    y_pred_ridge = Ridge_model.predict(x)
    r2_ridge = r2_score(y, y_pred_ridge)
    mse_ridge = mean_squared_error(y, y_pred_ridge)

    print(f"Ridge Regression : ")
    print("R2 score:", r2_ridge)
    print("MSE:", mse_ridge)
    print("Coefficients:", Ridge_model.coef_)
    print("Intercept:", Ridge_model.intercept_)
    print(" ")

    Lasso_model = Lasso(alpha)
```

```
    Lasso_model.fit(x, y)
    y_pred_lasso = Lasso_model.predict(x)
    r2_lasso = r2_score(y, y_pred_lasso)
    mse_lasso = mean_squared_error(y, y_pred_lasso)

    print(f"Lasso Regression :")
    print("R2 score:", r2_lasso)
    print("MSE:", mse_lasso)
    print("Coefficients:", Lasso_model.coef_)
    print("Intercept:", Lasso_model.intercept_)
    print(" ")
```

```
regression_model(x, y, alpha=0.01)
```

```
Ridge Regression :
R2 score: 0.44212736037527545
MSE: 114566617282.07559
Coefficients: [-6.92568839e-06 -2.07876255e+03 -9.66267102e+04  4.85676202e+05
 -8.77102882e+05  3.89593662e+04 -4.03184831e+04 -2.11235934e+03
 -4.97843034e+03  9.00574550e+02  1.65747221e+04  3.37623288e+02
  1.43521398e+04 -2.17977969e+03  2.05991458e+04  6.03551658e+03
 -3.58180794e+03  6.61011621e+05 -1.72325865e+04 -5.21369548e+04
  4.06365389e-04  8.04637835e+01  1.49217490e+05  1.78712265e+05
 -3.85975312e+04 -1.22322631e+05 -3.19188560e+02  3.47845142e+04
 -6.48503203e+01 -1.36533222e+04  1.83751116e+05 -4.02640940e+04
 -8.84104990e+04  1.94717058e+05]
Intercept: 186652671.95078948

/usr/local/lib/python3.12/dist-packages/scipy/_lib/_util.py:1233: LinAlgWarning: Ill-conditioned matrix (rcond=8.76175e-21): res
  return f(*arrays, *other_args, **kwargs)
Lasso Regression :
R2 score: 0.4421272999222039
MSE: 114566629696.92337
Coefficients: [-6.92595744e-06 -2.07880157e+03 -9.66007034e+04  4.85785205e+05
 -8.77420338e+05  3.75220209e+04 -3.88942599e+04 -2.11235157e+03
 -4.97973085e+03  9.02443380e+02  1.65813518e+04  3.37400066e+02
  1.43435355e+04 -2.17792869e+03  2.06163993e+04  6.04105824e+03
 -3.58776161e+03  6.61120608e+05 -1.72311687e+04 -5.17233626e+04
  4.06379505e-04  8.04652766e+01  1.49230914e+05  1.78707084e+05
 -3.86021595e+04 -1.22338732e+05 -3.18423668e+02  3.47868170e+04
 -6.48320749e+01 -1.36549080e+04  1.83754397e+05 -4.02620907e+04
 -8.84163729e+04  1.94715938e+05]
Intercept: 186652806.5040314

/usr/local/lib/python3.12/dist-packages/sklearn/linear_model/_coordinate_descent.py:695: ConvergenceWarning: Objective did not c
  model = cd_fast.enet_coordinate_descent(
```

```
regression_model(x, y, alpha=0.1)
```

```
/usr/local/lib/python3.12/dist-packages/scipy/_lib/_util.py:1233: LinAlgWarning: Ill-conditioned matrix (rcond=8.80875e-21): res
  return f(*arrays, *other_args, **kwargs)
Ridge Regression :
R2 score: 0.4421272357114049
MSE: 114566642883.47083
Coefficients: [-6.92354282e-06 -2.07807522e+03 -9.68484204e+04  4.84882012e+05
 -8.74323318e+05  3.88814181e+04 -4.02225641e+04 -2.11370876e+03
 -4.97619099e+03  9.00629452e+02  1.65831326e+04  3.37738146e+02
  1.43575430e+04 -2.17698323e+03  2.05405292e+04  6.03416642e+03
 -3.58595247e+03  6.60003753e+05 -1.72328369e+04 -5.21288379e+04
  4.06345739e-04  8.04638290e+01  1.48979089e+05  1.78680743e+05
 -3.85625235e+04 -1.22108034e+05 -3.19535568e+02  3.47409731e+04
 -6.48951718e+01 -1.36296518e+04  1.83750138e+05 -4.02813623e+04
 -8.83898811e+04  1.94764132e+05]
Intercept: 186507782.1898901

Lasso Regression :
R2 score: 0.4421272970473996
MSE: 114566630287.30295
Coefficients: [-6.92550045e-06 -2.07881990e+03 -9.65715954e+04  4.85732925e+05
 -8.77392161e+05  3.74898784e+04 -3.88618760e+04 -2.11238912e+03
 -4.97974570e+03  9.02501127e+02  1.65815778e+04  3.37394028e+02
  1.43409666e+04 -2.17781181e+03  2.06153437e+04  6.04095821e+03
 -3.58772357e+03  6.61097214e+05 -1.72309349e+04 -5.17128191e+04
  4.06379258e-04  8.04651489e+01  1.49212272e+05  1.78694779e+05
 -3.85994614e+04 -1.22329438e+05 -3.18758420e+02  3.47842678e+04
 -6.48296928e+01 -1.36541800e+04  1.83753987e+05 -4.02623095e+04
 -8.84152610e+04  1.94716538e+05]
```

```
Intercept: 186647827.6147018

/usr/local/lib/python3.12/dist-packages/sklearn/linear_model/_coordinate_descent.py:695: ConvergenceWarning: Objective did not c
  model = cd_fast.enet_coordinate_descent(
```

```
regression_model(x, y, alpha=1)
```

```
/usr/local/lib/python3.12/dist-packages/scipy/_lib/_util.py:1233: LinAlgWarning: Ill-conditioned matrix (rcond=9.27905e-21): res
  return f(*arrays, *other_args, **kwargs)
Ridge Regression :
R2 score: 0.4421154170307465
MSE: 114569070007.81355
Coefficients: [-6.90119283e-06 -2.07164010e+03 -9.87952312e+04  4.76905115e+05
 -8.47610734e+05  3.81170812e+04 -3.92853751e+04 -2.12682114e+03
 -4.95476528e+03  9.01336913e+02  1.66653899e+04  3.38839588e+02
  1.44098180e+04 -2.14972884e+03  1.99755045e+04  6.02124378e+03
 -3.62626351e+03  6.50084677e+05 -1.72363862e+04 -5.20479239e+04
  4.06157865e-04  8.04637199e+01  1.46629953e+05  1.78339282e+05
 -3.82260175e+04 -1.20014714e+05 -3.25542873e+02  3.43119551e+04
 -6.53091146e+01 -1.34033911e+04  1.83738962e+05 -4.04491005e+04
 -8.81880067e+04  1.95216075e+05]
Intercept: 185087004.99618247

Lasso Regression :
R2 score: 0.4421272559726087
MSE: 114566638722.5613
Coefficients: [-6.92093054e-06 -2.07900325e+03 -9.62805148e+04  4.85210122e+05
 -8.77110393e+05  3.71684533e+04 -3.85380368e+04 -2.11276463e+03
 -4.97989422e+03  9.03078603e+02  1.65838381e+04  3.37333650e+02
  1.43152780e+04 -2.17664301e+03  2.06047878e+04  6.03995797e+03
 -3.58734316e+03  6.60863279e+05 -1.72285959e+04 -5.16073845e+04
  4.06376783e-04  8.04638725e+01  1.49025844e+05  1.78571726e+05
 -3.85724804e+04 -1.22236494e+05 -3.22105942e+02  3.47587758e+04
 -6.48058716e+01 -1.36469006e+04  1.83749886e+05 -4.02644973e+04
 -8.84041416e+04  1.94722535e+05]
Intercept: 186598038.72140783

/usr/local/lib/python3.12/dist-packages/sklearn/linear_model/_coordinate_descent.py:695: ConvergenceWarning: Objective did not c
  model = cd_fast.enet_coordinate_descent(
```

```
#### 6. Support Vector Machine (SVM)
```

```
# TASK 6: SUPPORT VECTOR MACHINE (SVM)
print("TASK 6: SUPPORT VECTOR MACHINE (SVM)")

from sklearn.svm import SVC
from sklearn.model_selection import GridSearchCV
from sklearn.metrics import (
    accuracy_score,
    precision_score,
    recall_score,
    f1_score,
    roc_auc_score,
    confusion_matrix,
    roc_curve,
    precision_recall_curve
)
from sklearn.preprocessing import StandardScaler
from sklearn.model_selection import train_test_split
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns

X_svm = X_class.copy()
y_svm = y_class.copy()

X_train_svm, X_test_svm, y_train_svm, y_test_svm = train_test_split(
    X_svm, y_svm, test_size=0.2, random_state=42, stratify=y_svm
)

scaler_svm = StandardScaler()
X_train_svm_scaled = scaler_svm.fit_transform(X_train_svm)
X_test_svm_scaled = scaler_svm.transform(X_test_svm)

print(f"Training set: {X_train_svm_scaled.shape}")
```

```
print(f"Testing set: {X_test_svm_scaled.shape}")
print(f"Class distribution (train): {np.bincount(y_train_svm)}")
print(f"Class distribution (test): {np.bincount(y_test_svm)}")

print("\n SVM WITH DIFFERENT KERNELS")

kernels = ['linear', 'poly', 'rbf', 'sigmoid']
svm_results = []

for kernel in kernels:
    print(f"\nTraining SVM with {kernel} kernel...")

    if kernel == 'linear':
        svm = SVC(kernel=kernel, C=1.0, random_state=42, probability=True)
    elif kernel == 'poly':
        svm = SVC(kernel=kernel, C=1.0, degree=3, random_state=42, probability=True)
    else:
        svm = SVC(kernel=kernel, C=1.0, gamma='scale', random_state=42, probability=True)

    svm.fit(X_train_svm_scaled, y_train_svm)

    y_pred_svm = svm.predict(X_test_svm_scaled)
    y_prob_svm = svm.predict_proba(X_test_svm_scaled)[:, 1]

    accuracy = accuracy_score(y_test_svm, y_pred_svm)
    precision = precision_score(y_test_svm, y_pred_svm)
    recall = recall_score(y_test_svm, y_pred_svm)
    f1 = f1_score(y_test_svm, y_pred_svm)
    auc_score = roc_auc_score(y_test_svm, y_prob_svm)

    svm_results.append({
        'Kernel': kernel,
        'Accuracy': accuracy,
        'Precision': precision,
        'Recall': recall,
        'F1': f1,
        'AUC': auc_score,
        'Support_Vectors': len(svm.support_)
    })

    print(f"  Accuracy: {accuracy:.4f}")
    print(f"  F1-Score: {f1:.4f}")
    print(f"  AUC: {auc_score:.4f}")
    print(f"  Support Vectors: {len(svm.support_)}")

svm_results_df = pd.DataFrame(svm_results)
print("\nSVM Performance Summary:")
print(svm_results_df)

print("\n SVM HYPERPARAMETER TUNING")

param_grid = {
    'C': [0.1, 1, 10, 100],
    'gamma': ['scale', 'auto', 0.1, 0.01, 0.001],
    'kernel': ['rbf']
}

sample_size = min(1000, len(X_train_svm_scaled))
X_sample = X_train_svm_scaled[:sample_size]
y_sample = y_train_svm[:sample_size]

print("Performing Grid Search for RBF SVM (using sample)...")
grid_search = GridSearchCV(
    SVC(random_state=42, probability=True),
    param_grid,
    cv=3,
    scoring='f1',
    n_jobs=-1,
    verbose=1
)
grid_search.fit(X_sample, y_sample)

print(f"Best parameters: {grid_search.best_params_}")
print(f"Best cross-validation score: {grid_search.best_score_:.4f}")

best_svm = grid_search.best_estimator_
best_svm.fit(X_train_svm_scaled, y_train_svm)
```

```
y_pred_best_svm = best_svm.predict(X_test_svm_scaled)
y_prob_best_svm = best_svm.predict_proba(X_test_svm_scaled)[:, 1]

best_accuracy = accuracy_score(y_test_svm, y_pred_best_svm)
best_f1 = f1_score(y_test_svm, y_pred_best_svm)
best_auc = roc_auc_score(y_test_svm, y_prob_best_svm)

print(f"\nBest SVM Performance:")
print(f"  Accuracy: {best_accuracy:.4f}")
print(f"  F1-Score: {best_f1:.4f}")
print(f"  AUC: {best_auc:.4f}")

print("\nSUPPORT VECTORS ANALYSIS")

svm_linear = SVC(kernel='linear', C=1.0, random_state=42)
svm_linear.fit(X_train_svm_scaled[:, :2], y_train_svm)  # Use 2D for visualization

print(f"Linear SVM Support Vectors: {len(svm_linear.support_vectors_)}")
print(f"Number of support vectors per class: {np.bincount(y_train_svm.iloc[svm_linear.support_])}")

# Visualization
fig, axes = plt.subplots(2, 3, figsize=(18, 12))

# Kernel Performance Comparison
x_pos = np.arange(len(kernels))
width = 0.2

axes[0, 0].bar(x_pos - 1.5*width, svm_results_df['Accuracy'], width, label='Accuracy')
axes[0, 0].bar(x_pos - 0.5*width, svm_results_df['Precision'], width, label='Precision')
axes[0, 0].bar(x_pos + 0.5*width, svm_results_df['Recall'], width, label='Recall')
axes[0, 0].bar(x_pos + 1.5*width, svm_results_df['F1'], width, label='F1')
axes[0, 0].set_title('SVM Kernel Performance Comparison')
axes[0, 0].set_xlabel('Kernel')
axes[0, 0].set_ylabel('Score')
axes[0, 0].set_xticks(x_pos)
axes[0, 0].set_xticklabels(kernels)
axes[0, 0].legend()
axes[0, 0].grid(True, axis='y')

# Support Vectors per Kernel
axes[0, 1].bar(kernels, svm_results_df['Support_Vectors'])
axes[0, 1].set_title('Number of Support Vectors per Kernel')
axes[0, 1].set_xlabel('Kernel')
axes[0, 1].set_ylabel('Number of Support Vectors')
axes[0, 1].grid(True, axis='y')

# Confusion Matrix for Best SVM
cm_best_svm = confusion_matrix(y_test_svm, y_pred_best_svm)
sns.heatmap(cm_best_svm, annot=True, fmt='d', cmap='YlOrRd', ax=axes[0, 2])
axes[0, 2].set_title(f'Best SVM ({grid_search.best_params_["kernel"]}) Confusion Matrix')
axes[0, 2].set_xlabel('Predicted')
axes[0, 2].set_ylabel('Actual')

# ROC Curves for Different Kernels
for idx, kernel in enumerate(kernels):
    svm_kernel = SVC(kernel=kernel, C=1.0, probability=True, random_state=42)
    svm_kernel.fit(X_train_svm_scaled, y_train_svm)
    y_prob_kernel = svm_kernel.predict_proba(X_test_svm_scaled)[:, 1]
    fpr, tpr, _ = roc_curve(y_test_svm, y_prob_kernel)
    auc_kernel = roc_auc_score(y_test_svm, y_prob_kernel)
    axes[1, 0].plot(fpr, tpr, label=f'{kernel} (AUC={auc_kernel:.3f})')

axes[1, 0].plot([0, 1], [0, 1], 'k--', label='Random')
axes[1, 0].set_title('SVM ROC Curves by Kernel')
axes[1, 0].set_xlabel('False Positive Rate')
axes[1, 0].set_ylabel('True Positive Rate')
axes[1, 0].legend()
axes[1, 0].grid(True)

# Decision Boundary Visualization (2D)
def plot_decision_boundary(clf, X, y, ax, title):
    """Plot decision boundary for 2D data"""
    # Create mesh grid
    x_min, x_max = X[:, 0].min() - 1, X[:, 0].max() + 1
    y_min, y_max = X[:, 1].min() - 1, X[:, 1].max() + 1
    xx, yy = np.meshgrid(np.arange(x_min, x_max, 0.1),
```

```
                         np.arange(y_min, y_max, 0.1))

    # Predict for each point in mesh
    Z = clf.predict(np.c_[xx.ravel(), yy.ravel()])
    Z = Z.reshape(xx.shape)

    # Plot decision boundary and margins
    ax.contourf(xx, yy, Z, alpha=0.4, cmap='coolwarm')
    ax.scatter(X[:, 0], X[:, 1], c=y, s=30, edgecolor='k', cmap='coolwarm')

    # Plot support vectors
    if hasattr(clf, 'support_vectors_'):
        ax.scatter(clf.support_vectors_[:, 0], clf.support_vectors_[:, 1],
                   s=100, facecolors='none', edgecolors='k', linewidths=2,
                   label='Support Vectors')

    ax.set_xlabel('Feature 1')
    ax.set_ylabel('Feature 2')
    ax.set_title(title)
    ax.legend()

X_2d = X_train_svm_scaled[:, :2]

svm_linear_2d = SVC(kernel='linear', C=1.0, random_state=42)
svm_linear_2d.fit(X_2d, y_train_svm)
plot_decision_boundary(svm_linear_2d, X_2d, y_train_svm, axes[1, 1], 'Linear SVM Decision Boundary')

svm_rbf_2d = SVC(kernel='rbf', C=1.0, gamma='scale', random_state=42)
svm_rbf_2d.fit(X_2d, y_train_svm)
plot_decision_boundary(svm_rbf_2d, X_2d, y_train_svm, axes[1, 2], 'RBF SVM Decision Boundary')

plt.tight_layout()
plt.show()

# Margin Analysis
print("\n MARGIN ANALYSIS FOR LINEAR SVM")

if hasattr(svm_linear, 'coef_'):
    w = svm_linear.coef_[0]
    margin = 2 / np.sqrt(np.sum(w ** 2))
    print(f"Margin width: {margin:.4f}")

    if len(svm_linear.support_vectors_) > 0:
        distances = np.abs(np.dot(svm_linear.support_vectors_, w) + svm_linear.intercept_[0]) / np.linalg.norm(w)
        print(f"Support vector distances to hyperplane:")
        print(f"  Min: {distances.min():.4f}")
        print(f"  Max: {distances.max():.4f}")
        print(f"  Mean: {distances.mean():.4f}")
```

```
TASK 6: SUPPORT VECTOR MACHINE (SVM)
Training set: (12136, 10)
Testing set: (3035, 10)
Class distribution (train): [6096 6040]
Class distribution (test): [1525 1510]

 SVM WITH DIFFERENT KERNELS

Training SVM with linear kernel...
  Accuracy: 0.7242
  F1-Score: 0.7171
  AUC: 0.7914
  Support Vectors: 7723

Training SVM with poly kernel...
  Accuracy: 0.7868
  F1-Score: 0.7853
  AUC: 0.8472
  Support Vectors: 7493

Training SVM with rbf kernel...
  Accuracy: 0.8043
  F1-Score: 0.8036
  AUC: 0.8770
  Support Vectors: 6009

Training SVM with sigmoid kernel...
  Accuracy: 0.6053
  F1-Score: 0.6051
  AUC: 0.6144
  Support Vectors: 5013

SVM Performance Summary:
    Kernel  Accuracy  Precision    Recall        F1       AUC  Support_Vectors
0   linear  0.724217   0.732229  0.702649  0.717134  0.791447             7723
1     poly  0.786820   0.787092  0.783444  0.785264  0.847234             7493
2      rbf  0.804283   0.802510  0.804636  0.803571  0.877005             6009
3  sigmoid  0.605272   0.602362  0.607947  0.605142  0.614377             5013

 SVM HYPERPARAMETER TUNING
Performing Grid Search for RBF SVM (using sample)...
Fitting 3 folds for each of 20 candidates, totalling 60 fits
Best parameters: {'C': 100, 'gamma': 0.01, 'kernel': 'rbf'}
Best cross-validation score: 0.7706
```

```
###################### 7. Classifier Models: Naïve Bayes, K-Nearest Neighbors (KNN), Decision Trees, Random Forests
  Accuracy: 0.7990
```

```
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
from sklearn.naive_bayes import GaussianNB
from sklearn.neighbors import KNeighborsClassifier
from sklearn.tree import DecisionTreeClassifier
from sklearn.ensemble import RandomForestClassifier
from sklearn.metrics import accuracy_score, precision_score, recall_score, f1_score, confusion_matrix


# Define the actual target column name for consistency
target = "latestPrice"

# Classification target: above median price
median_price = df[target].median()
df["y"] = (df[target] > median_price).astype(int)

# Numeric features only
num_cols = df.select_dtypes(include=[np.number]).columns.tolist()
num_cols.remove(target)
num_cols.remove("y")
X = df[num_cols]
y = df["y"]

# Train-test split
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.25, random_state=42, stratify=y
)

# Standardize numeric features
saler = StandardScaler()
X_train_scaled = saler.fit_transform(X_train)
X_test_scaled = saler.transform(X_test)
```

```
# 1. Naive Bayes
model_nb = GaussianNB()
model_nb.fit(X_train_scaled, y_train)
y_pred_nb = model_nb.predict(X_test_scaled)
acc_nb = accuracy_score(y_test, y_pred_nb)
print("Naive Bayes:")
print("Accuracy:", acc_nb)
print(" ")

# 2.. K-Nearest Neighbors
model_knn = KNeighborsClassifier(n_neighbors=3)
model_knn.fit(X_train_scaled, y_train)
y_pred_knn = model_knn.predict(X_test_scaled)
acc_knn = accuracy_score(y_test, y_pred_knn)
print("KNN:")
print("Accuracy:", acc_knn)
print(" ")

# 3.. Decision Tree
model_dt = DecisionTreeClassifier(random_state=0)
model_dt.fit(X_train_scaled, y_train)
y_pred_dt = model_dt.predict(X_test_scaled)
acc_dt = accuracy_score(y_test, y_pred_dt)
print("Decision Tree:")
print("Accuracy:", acc_dt)
print(" ")

# 4.. Random Forest
model_rf = RandomForestClassifier(n_estimators=10, random_state=0)
model_rf.fit(X_train_scaled, y_train)
y_pred_rf = model_rf.predict(X_test_scaled)
acc_rf = accuracy_score(y_test, y_pred_rf)
print("Random Forest:")
print("Accuracy:", acc_rf)
print(" ")
```

```
Naive Bayes:
Accuracy: 0.6351173213814922

KNN:
Accuracy: 0.7977853941471131

Decision Tree:
Accuracy: 0.8151858687055101

Random Forest:
Accuracy: 0.8634326390719747
```

```
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
from sklearn.naive_bayes import GaussianNB
from sklearn.neighbors import KNeighborsClassifier
from sklearn.tree import DecisionTreeClassifier
from sklearn.ensemble import RandomForestClassifier
from sklearn.metrics import accuracy_score, precision_score, recall_score, f1_score, confusion_matrix


target = "latestPrice"
median_price = df[target].median()
df["y"] = (df[target] > median_price).astype(int)

# Feature selection (first 10 numeric columns)
numerical_cols = df.select_dtypes(include=[np.number]).columns.tolist()
numerical_cols.remove(target)
numerical_cols.remove("y")

X = df[numerical_cols[:10]].fillna(df[numerical_cols[:10]].median())
y = df["y"]

print(f"X shape: {X.shape}")
print(f"y shape: {y.shape}")
print(f"Classes: {np.bincount(y)}")


X_train, X_test, y_train, y_test = train_test_split(
```

```
    X, y, test_size=0.2, random_state=42, stratify=y
)

scaler = StandardScaler()
X_train_scaled = scaler.fit_transform(X_train)
X_test_scaled = scaler.transform(X_test)

print(f"\nTraining set: {X_train_scaled.shape}")
print(f"Testing set: {X_test_scaled.shape}")


classifiers = {
    'Naive Bayes': GaussianNB(),
    'K-Nearest Neighbors': KNeighborsClassifier(n_neighbors=5),
    'Decision Tree': DecisionTreeClassifier(random_state=42),
    'Random Forest': RandomForestClassifier(n_estimators=100, random_state=42)
}

results = []
for name, clf in classifiers.items():
    print(f"\nTraining {name}...")
    clf.fit(X_train_scaled, y_train)
    y_pred = clf.predict(X_test_scaled)
    y_pred_train = clf.predict(X_train_scaled)

    acc_train = accuracy_score(y_train, y_pred_train)
    acc_test = accuracy_score(y_test, y_pred)
    prec = precision_score(y_test, y_pred, zero_division=0)
    rec = recall_score(y_test, y_pred, zero_division=0)
    f1 = f1_score(y_test, y_pred, zero_division=0)

    results.append({
        'Classifier': name,
        'Train_Accuracy': acc_train,
        'Test_Accuracy': acc_test,
        'Precision': prec,
        'Recall': rec,
        'F1_Score': f1
    })

    print(f"  Training Accuracy: {acc_train:.4f}")
    print(f"  Testing Accuracy: {acc_test:.4f}")
    print(f"  Precision: {prec:.4f}")
    print(f"  Recall: {rec:.4f}")
    print(f"  F1-Score: {f1:.4f}")

    # Overfitting check
    if acc_train - acc_test > 0.1:
        print(f"  ⚠️ Potential overfitting: {acc_train - acc_test:.4f}")

results_df = pd.DataFrame(results)
print("\nPerformance Summary:")
print(results_df.to_string(index=False))

# Visualization
fig, axes = plt.subplots(2, 3, figsize=(18, 10))

# Accuracy comparison
x_pos = np.arange(len(classifiers))
width = 0.35
axes[0, 0].bar(x_pos - width/2, results_df['Train_Accuracy'], width, label='Train')
axes[0, 0].bar(x_pos + width/2, results_df['Test_Accuracy'], width, label='Test')
axes[0, 0].set_xticks(x_pos)
axes[0, 0].set_xticklabels(results_df['Classifier'], rotation=45)
axes[0, 0].set_ylabel('Accuracy')
axes[0, 0].set_title('Training vs Testing Accuracy')
axes[0, 0].legend()
axes[0, 0].grid(True, axis='y')

# F1-Score comparison
axes[0, 1].bar(results_df['Classifier'], results_df['F1_Score'])
axes[0, 1].set_xticklabels(results_df['Classifier'], rotation=45)
axes[0, 1].set_ylabel('F1-Score')
axes[0, 1].set_title('F1-Score Comparison')
axes[0, 1].grid(True, axis='y')

# Precision-Recall trade-off
axes[0, 2].scatter(results_df['Precision'], results_df['Recall'], s=100)
```

```
axes[0, 2].scatter(results_df['Precision'], results_df['Recall'], s=100)
for i, row in results_df.iterrows():
    axes[0, 2].annotate(row['Classifier'], (row['Precision'], row['Recall']))
axes[0, 2].set_xlabel('Precision')
axes[0, 2].set_ylabel('Recall')
axes[0, 2].set_title('Precision-Recall Trade-off')
axes[0, 2].grid(True)

# Confusion Matrices (first 3 classifiers)
for idx, (name, clf) in enumerate(list(classifiers.items())[:3]):
    clf.fit(X_train_scaled, y_train)
    y_pred = clf.predict(X_test_scaled)
    cm = confusion_matrix(y_test, y_pred)
    sns.heatmap(cm, annot=True, fmt='d', cmap='Blues', ax=axes[1, idx])
    axes[1, idx].set_title(f'{name} Confusion Matrix')
    axes[1, idx].set_xlabel('Predicted')
    axes[1, idx].set_ylabel('Actual')

plt.tight_layout()
plt.show()
```

```
############################################################  8. Single-Layer and Multi-Layer Perceptron Models (Neural Network
```

```
# TASK 8: NEURAL NETWORKS - FIXED VERSION
print("TASK 8: NEURAL NETWORKS")

from sklearn.neural_network import MLPClassifier
from tensorflow import keras
from tensorflow.keras import models, layers

# 8.1 Single-Layer Perceptron (from scratch)
print("\n8.1 SINGLE-LAYER PERCEPTRON (FROM SCRATCH)")

class SingleLayerPerceptron:
    def __init__(self, learning_rate=0.01, epochs=100):
        self.lr = learning_rate
        self.epochs = epochs
        self.weights = None
        self.bias = None

    def activation(self, x):
        return np.where(x >= 0, 1, 0)

    def fit(self, X, y):
        n_samples, n_features = X.shape
        self.weights = np.zeros(n_features)
        self.bias = 0

        for epoch in range(self.epochs):
            for idx in range(n_samples):
                linear = np.dot(X[idx], self.weights) + self.bias
                y_pred = self.activation(linear)

                update = self.lr * (y.iloc[idx] - y_pred)
                self.weights += update * X[idx]
                self.bias += update

            if epoch % 20 == 0:
```

```
                accuracy = self.score(X, y)
                print(f"Epoch {epoch}: Accuracy = {accuracy:.4f}")

        return self

    def predict(self, X):
        linear = np.dot(X, self.weights) + self.bias
        return self.activation(linear)

    def score(self, X, y):
        y_pred = self.predict(X)
        return accuracy_score(y, y_pred)

print("Training Single-Layer Perceptron from scratch...")
slp = SingleLayerPerceptron(learning_rate=0.1, epochs=100)
slp.fit(X_train_scaled, y_train)
slp_acc = slp.score(X_test_scaled, y_test)
print(f"Test Accuracy (from scratch): {slp_acc:.4f}")

# 8.2 Multi-Layer Perceptron (MLP) using scikit-learn
print("\n8.2 MULTI-LAYER PERCEPTRON (MLP)")

mlp_simple = MLPClassifier(hidden_layer_sizes=(10,),
                           max_iter=1000,
                           random_state=42)
mlp_simple.fit(X_train_scaled, y_train)
mlp_simple_acc = mlp_simple.score(X_test_scaled, y_test)
print(f"Simple MLP (10 neurons): Accuracy = {mlp_simple_acc:.4f}")

mlp_deep = MLPClassifier(hidden_layer_sizes=(64, 32, 16),
                         max_iter=1000,
                         random_state=42,
                         learning_rate_init=0.001)
mlp_deep.fit(X_train_scaled, y_train)
mlp_deep_acc = mlp_deep.score(X_test_scaled, y_test)
print(f"Deep MLP (64-32-16): Accuracy = {mlp_deep_acc:.4f}")

# 8.3 CNN for comparison (using Keras) - FIXED VERSION
print("\n8.3 CONVOLUTIONAL NEURAL NETWORK (CNN) - FOR TABULAR DATA")

# Reshape data for 1D CNN (more suitable for tabular data)
n_samples = X_train_scaled.shape[0]
n_features = X_train_scaled.shape[1]

X_train_cnn = X_train_scaled.reshape(-1, n_features, 1)
X_test_cnn = X_test_scaled.reshape(-1, n_features, 1)

print(f"Reshaped to: {X_train_cnn.shape}")

# Build 1D CNN model (more appropriate for tabular data)
cnn_model = models.Sequential([
    layers.Conv1D(32, kernel_size=3, activation='relu', input_shape=(n_features, 1)),
    layers.MaxPooling1D(pool_size=2),
    layers.Conv1D(64, kernel_size=3, activation='relu'),
    layers.MaxPooling1D(pool_size=2),
    layers.Flatten(),
    layers.Dense(64, activation='relu'),
    layers.Dropout(0.5),
    layers.Dense(1, activation='sigmoid')
])

cnn_model.compile(optimizer='adam',
                  loss='binary_crossentropy',
                  metrics=['accuracy'])

cnn_model.summary()

print("\nTraining 1D CNN...")
history = cnn_model.fit(X_train_cnn, y_train,
                        epochs=20,
                        batch_size=32,
                        validation_split=0.2,
                        verbose=1)

cnn_loss, cnn_acc = cnn_model.evaluate(X_test_cnn, y_test, verbose=0)
print(f"1D CNN Test Accuracy: {cnn_acc:.4f}")
```

```
print("\n8.4 ANN vs CNN COMPARISON")
print("-" * 40)

nn_results = pd.DataFrame({
    'Model': ['Single-Layer Perceptron', 'Simple MLP', 'Deep MLP', '1D CNN'],
    'Accuracy': [slp_acc, mlp_simple_acc, mlp_deep_acc, cnn_acc],
    'Architecture': ['1 layer', '10 neurons', '64-32-16', 'Conv1D(32)-Conv1D(64)']
})

print("Neural Network Comparison:")
print(nn_results.to_string(index=False))

print("\nKey Differences:")
print("1. Single-Layer: Linear separation only, cannot learn XOR")
print("2. MLP: Can learn complex non-linear patterns")
print("3. 1D CNN: Better for sequential/tabular data, captures local patterns")
print("4. For tabular data, MLP often outperforms CNN but CNN can capture local dependencies")

# Visualization
fig, axes = plt.subplots(2, 2, figsize=(12, 10))

# Accuracy comparison
axes[0, 0].bar(nn_results['Model'], nn_results['Accuracy'])
axes[0, 0].set_xticklabels(nn_results['Model'], rotation=45, ha='right')
axes[0, 0].set_ylabel('Accuracy')
axes[0, 0].set_title('Neural Network Accuracy Comparison')
axes[0, 0].grid(True, axis='y')

# MLP loss curve
if hasattr(mlp_deep, 'loss_curve_'):
    axes[0, 1].plot(mlp_deep.loss_curve_)
    axes[0, 1].set_xlabel('Iteration')
    axes[0, 1].set_ylabel('Loss')
    axes[0, 1].set_title('MLP Training Loss')
    axes[0, 1].grid(True)

# CNN training history
if 'history' in locals():
    axes[1, 0].plot(history.history['accuracy'], label='Train')
    axes[1, 0].plot(history.history['val_accuracy'], label='Validation')
    axes[1, 0].set_xlabel('Epoch')
    axes[1, 0].set_ylabel('Accuracy')
    axes[1, 0].set_title('1D CNN Training History')
    axes[1, 0].legend()
    axes[1, 0].grid(True)

# Model architecture comparison
arch_labels = ['SLP', 'MLP-Simple', 'MLP-Deep', '1D CNN']
arch_params = [
    1 * n_features + 1,  # SLP: n_weights + 1_bias
    10 * (n_features + 1),  # Simple MLP
    64*32 + 32*16 + 16*1,  # Deep MLP (approx)
    32*3*1 + 64*3*32  # CNN params (approx)
]
axes[1, 1].bar(arch_labels, arch_params)
axes[1, 1].set_xlabel('Model')
axes[1, 1].set_ylabel('Parameter Count (approx)')
axes[1, 1].set_title('Model Complexity Comparison')
axes[1, 1].grid(True, axis='y')
axes[1, 1].ticklabel_format(style='sci', axis='y', scilimits=(0,0))

plt.tight_layout()
plt.show()
```

```
TASK 8: NEURAL NETWORKS

8.1 SINGLE-LAYER PERCEPTRON (FROM SCRATCH)
Training Single-Layer Perceptron from scratch...
Epoch 0: Accuracy = 0.5958
Epoch 20: Accuracy = 0.6163
Epoch 40: Accuracy = 0.5864
Epoch 60: Accuracy = 0.6332
Epoch 80: Accuracy = 0.6066
Test Accuracy (from scratch): 0.6316

8.2 MULTI-LAYER PERCEPTRON (MLP)
Simple MLP (10 neurons): Accuracy = 0.8072
Deep MLP (64-32-16): Accuracy = 0.8178

8.3 CONVOLUTIONAL NEURAL NETWORK (CNN) - FOR TABULAR DATA
Reshaped to: (12136, 10, 1)
/usr/local/lib/python3.12/dist-packages/keras/src/layers/convolutional/base_conv.py:113: UserWarning: Do not pass an `input_shap
  super().__init__(activity_regularizer=activity_regularizer, **kwargs)
Model: "sequential"
```

Layer (type)	Output Shape	Param #
conv1d (Conv1D)	(None, 8, 32)	128
max_pooling1d (MaxPooling1D)	(None, 4, 32)	0
conv1d_1 (Conv1D)	(None, 2, 64)	6,208
max_pooling1d_1 (MaxPooling1D)	(None, 1, 64)	0
flatten (Flatten)	(None, 64)	0
dense (Dense)	(None, 64)	4,160
dropout (Dropout)	(None, 64)	0
dense_1 (Dense)	(None, 1)	65

```
Total params: 10,561 (41.25 KB)
Trainable params: 10,561 (41.25 KB)
Non-trainable params: 0 (0.00 B)

Training 1D CNN...
Epoch 1/20
304/304 ━━━━━━━━━━━━━━━━━━━━ 2s 2ms/step - accuracy: 0.6336 - loss: 0.6330 - val_accuracy: 0.7974 - val_loss: 0.4643
Epoch 2/20
304/304 ━━━━━━━━━━━━━━━━━━━━ 1s 2ms/step - accuracy: 0.7855 - loss: 0.4739 - val_accuracy: 0.8023 - val_loss: 0.4435
Epoch 3/20
304/304 ━━━━━━━━━━━━━━━━━━━━ 1s 3ms/step - accuracy: 0.7953 - loss: 0.4573 - val_accuracy: 0.8101 - val_loss: 0.4298
Epoch 4/20
304/304 ━━━━━━━━━━━━━━━━━━━━ 1s 3ms/step - accuracy: 0.8089 - loss: 0.4358 - val_accuracy: 0.8072 - val_loss: 0.4215
Epoch 5/20
304/304 ━━━━━━━━━━━━━━━━━━━━ 1s 2ms/step - accuracy: 0.8104 - loss: 0.4339 - val_accuracy: 0.8122 - val_loss: 0.4199
Epoch 6/20
304/304 ━━━━━━━━━━━━━━━━━━━━ 1s 2ms/step - accuracy: 0.8140 - loss: 0.4220 - val_accuracy: 0.8023 - val_loss: 0.4358
Epoch 7/20
304/304 ━━━━━━━━━━━━━━━━━━━━ 1s 2ms/step - accuracy: 0.8156 - loss: 0.4145 - val_accuracy: 0.8101 - val_loss: 0.4219
Epoch 8/20
304/304 ━━━━━━━━━━━━━━━━━━━━ 1s 2ms/step - accuracy: 0.8050 - loss: 0.4262 - val_accuracy: 0.8171 - val_loss: 0.4145
Epoch 9/20
304/304 ━━━━━━━━━━━━━━━━━━━━ 1s 2ms/step - accuracy: 0.8079 - loss: 0.4266 - val_accuracy: 0.8044 - val_loss: 0.4256
Epoch 10/20
304/304 ━━━━━━━━━━━━━━━━━━━━ 1s 2ms/step - accuracy: 0.8108 - loss: 0.4223 - val_accuracy: 0.8027 - val_loss: 0.4167
Epoch 11/20
304/304 ━━━━━━━━━━━━━━━━━━━━ 1s 2ms/step - accuracy: 0.8182 - loss: 0.4134 - val_accuracy: 0.8114 - val_loss: 0.4071
Epoch 12/20
304/304 ━━━━━━━━━━━━━━━━━━━━ 1s 2ms/step - accuracy: 0.8165 - loss: 0.4181 - val_accuracy: 0.8126 - val_loss: 0.4186
Epoch 13/20
304/304 ━━━━━━━━━━━━━━━━━━━━ 1s 2ms/step - accuracy: 0.8065 - loss: 0.4281 - val_accuracy: 0.8134 - val_loss: 0.4041
Epoch 14/20
304/304 ━━━━━━━━━━━━━━━━━━━━ 1s 3ms/step - accuracy: 0.8156 - loss: 0.4132 - val_accuracy: 0.8093 - val_loss: 0.4152
Epoch 15/20
304/304 ━━━━━━━━━━━━━━━━━━━━ 1s 3ms/step - accuracy: 0.8134 - loss: 0.4267 - val_accuracy: 0.8130 - val_loss: 0.4088
```

```
######################################## 9. Model Evaluation and Validation############################################
```

```
print("TASK 9: MODEL EVALUATION & VALIDATION")
print("\n9.1 K-FOLD CROSS-VALIDATION")

from sklearn.model_selection import KFold, cross_val_score
from sklearn.ensemble import RandomForestClassifier
from sklearn.neural_network import MLPClassifier
```

```
from sklearn.neighbors import KNeighborsClassifier
from sklearn.metrics import accuracy_score, precision_score, recall_score, f1_score
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns
from scipy import stats

best_classifiers = {
    'Random Forest': RandomForestClassifier(n_estimators=100, random_state=42),
    'MLP': MLPClassifier(hidden_layer_sizes=(64, 32), max_iter=1000, random_state=42),
    'KNN': KNeighborsClassifier(n_neighbors=5)
}

k = 5
kf = KFold(n_splits=k, shuffle=True, random_state=42)

cv_results = {}
for name, clf in best_classifiers.items():
    print(f"\n{name}:")

    # Use X, y from previous classification task
    cv_scores = cross_val_score(clf, X, y, cv=kf, scoring='accuracy')

    # Fit on training data for test set evaluation
    clf.fit(X_train_scaled, y_train)
    y_pred = clf.predict(X_test_scaled)

    acc = accuracy_score(y_test, y_pred)
    prec = precision_score(y_test, y_pred, zero_division=0)
    rec = recall_score(y_test, y_pred, zero_division=0)
    f1 = f1_score(y_test, y_pred, zero_division=0)

    cv_results[name] = {
        'CV_Scores': cv_scores,
        'CV_Mean': cv_scores.mean(),
        'CV_Std': cv_scores.std(),
        'Test_Accuracy': acc,
        'Test_Precision': prec,
        'Test_Recall': rec,
        'Test_F1': f1
    }

    print(f"  CV Scores: {cv_scores}")
    print(f"  CV Mean: {cv_scores.mean():.4f} (+/- {cv_scores.std() * 2:.4f})")
    print(f"  Test Accuracy: {acc:.4f}")

print("\n9.2 COMPREHENSIVE EVALUATION METRICS")

eval_df = pd.DataFrame({
    'Model': list(cv_results.keys()),
    'CV_Mean_Accuracy': [cv_results[m]['CV_Mean'] for m in cv_results],
    'CV_Std': [cv_results[m]['CV_Std'] for m in cv_results],
    'Test_Accuracy': [cv_results[m]['Test_Accuracy'] for m in cv_results],
    'Test_Precision': [cv_results[m]['Test_Precision'] for m in cv_results],
    'Test_Recall': [cv_results[m]['Test_Recall'] for m in cv_results],
    'Test_F1': [cv_results[m]['Test_F1'] for m in cv_results]
})

print("Evaluation Metrics Comparison:")
print(eval_df.to_string(index=False))

print("\n9.3 STATISTICAL SIGNIFICANCE TEST (Paired t-test)")

rf_scores = cv_results['Random Forest']['CV_Scores']
mlp_scores = cv_results['MLP']['CV_Scores']

t_stat, p_value = stats.ttest_rel(rf_scores, mlp_scores)
print(f"Random Forest vs MLP:")
print(f"  t-statistic: {t_stat:.4f}")
print(f"  p-value: {p_value:.4f}")
if p_value < 0.05:
    print("  Result: Statistically significant difference (p < 0.05)")
else:
    print("  Result: No statistically significant difference")

# 9.4 Visualization
```

```
fig, axes = plt.subplots(2, 2, figsize=(12, 10))

models = list(cv_results.keys())
x_pos = np.arange(len(models))
width = 0.35

cv_means = [cv_results[m]['CV_Mean'] for m in models]
test_accs = [cv_results[m]['Test_Accuracy'] for m in models]

axes[0, 0].bar(x_pos - width/2, cv_means, width, label='CV Mean')
axes[0, 0].bar(x_pos + width/2, test_accs, width, label='Test Accuracy')
axes[0, 0].set_xticks(x_pos)
axes[0, 0].set_xticklabels(models, rotation=45)
axes[0, 0].set_ylabel('Accuracy')
axes[0, 0].set_title('Cross-Validation vs Test Accuracy')
axes[0, 0].legend()
axes[0, 0].grid(True, axis='y')

# Box plot of CV scores
cv_data = [cv_results[m]['CV_Scores'] for m in models]
axes[0, 1].boxplot(cv_data, labels=models)
axes[0, 1].set_xticklabels(models, rotation=45)
axes[0, 1].set_ylabel('Accuracy')
axes[0, 1].set_title('Cross-Validation Score Distribution')
axes[0, 1].grid(True, axis='y')

# Radar chart for metrics
metrics = ['Accuracy', 'Precision', 'Recall', 'F1']
n_metrics = len(metrics)
angles = np.linspace(0, 2 * np.pi, n_metrics, endpoint=False).tolist()
angles += angles[:1]  # Close the polygon

for idx, model in enumerate(models[:3]):  # Plot first 3 models
    values = [
        cv_results[model]['Test_Accuracy'],
        cv_results[model]['Test_Precision'],
        cv_results[model]['Test_Recall'],
        cv_results[model]['Test_F1']
    ]
    values += values[:1]  # Close the polygon

    axes[1, 0].plot(angles, values, 'o-', label=model)
    axes[1, 0].fill(angles, values, alpha=0.25)

axes[1, 0].set_xticks(angles[:-1])
axes[1, 0].set_xticklabels(metrics)
axes[1, 0].set_title('Model Performance Radar Chart')
axes[1, 0].legend(loc='upper right')
axes[1, 0].grid(True)

# Error analysis
best_model = eval_df.loc[eval_df['Test_F1'].idxmax(), 'Model']
clf_best = best_classifiers[best_model]
clf_best.fit(X_train_scaled, y_train)
y_pred_best = clf_best.predict(X_test_scaled)

# Get misclassified samples
misclassified = np.where(y_pred_best != y_test)[0]
if len(misclassified) > 0:
    sample_idx = misclassified[0]
    axes[1, 1].bar(range(len(X_test_scaled[sample_idx])), X_test_scaled[sample_idx])
    axes[1, 1].set_xlabel('Feature Index')
    axes[1, 1].set_ylabel('Feature Value (scaled)')
    axes[1, 1].set_title(f'Misclassified Sample (True: {y_test.iloc[sample_idx]}, Pred: {y_pred_best[sample_idx]})')
    axes[1, 1].grid(True, axis='y')
else:
    axes[1, 1].text(0.5, 0.5, 'No misclassified samples',
                    ha='center', va='center', transform=axes[1, 1].transAxes)
    axes[1, 1].set_title('Error Analysis')

plt.tight_layout()
plt.show()
```

```
TASK 9: MODEL EVALUATION & VALIDATION

9.1 K-FOLD CROSS-VALIDATION

Random Forest:
  CV Scores: [0.82227842 0.80685564 0.82201714 0.82222552 0.82620101]
```

```
############################### 10. Unsupervised Learning: Clustering Analysis (Open-Ended Lab) ###############################
```

```
# TASK 10: CLUSTERING ANALYSIS

from sklearn.cluster import KMeans, AgglomerativeClustering
from sklearn.preprocessing import StandardScaler
from sklearn.decomposition import PCA
from sklearn.metrics import silhouette_score
from scipy.cluster.hierarchy import linkage, dendrogram

print("\n10.1 DATA PREPARATION FOR CLUSTERING")

numerical_cols_for_clustering = df.select_dtypes(include=[np.number]).columns.tolist()
if 'latestPrice' in numerical_cols_for_clustering:
    numerical_cols_for_clustering.remove('latestPrice')

X_cluster = df[numerical_cols_for_clustering].fillna(df[numerical_cols_for_clustering].median())

scaler_cluster = StandardScaler()
X_cluster_scaled = scaler_cluster.fit_transform(X_cluster)

print(f"Original features for clustering: {len(numerical_cols_for_clustering)}")
print(f"Scaled data shape for clustering: {X_cluster_scaled.shape}")

# Reduce dimensionality for visualization (if needed, e.g., for 2D plots)
pca = PCA(n_components=2)
X_cluster_pca = pca.fit_transform(X_cluster_scaled)

# 10.2 K-Means Clustering
print("\n10.2 K-MEANS CLUSTERING")


# Determine optimal k using Elbow Method and Silhouette Score
range_n_clusters = range(2, 10)
inertias = []
silhouette_scores = []

for n_clusters in range_n_clusters:
    kmeans = KMeans(n_clusters=n_clusters, random_state=42, n_init=10)
    kmeans.fit(X_cluster_scaled)
    inertias.append(kmeans.inertia_)

    if n_clusters > 1 and X_cluster_scaled.shape[0] > n_clusters:
        silhouette_scores.append(silhouette_score(X_cluster_scaled, kmeans.labels_))
    else:
        silhouette_scores.append(0) # or some placeholder for single cluster cases

# Plot Elbow Method
plt.figure(figsize=(12, 5))
plt.subplot(1, 2, 1)
plt.plot(range_n_clusters, inertias, marker='o')
plt.title('Elbow Method for K-Means')
plt.xlabel('Number of Clusters (k)')
plt.ylabel('Inertia')
plt.grid(True)

# Plot Silhouette Scores
plt.subplot(1, 2, 2)
plt.plot(range_n_clusters, silhouette_scores, marker='o', color='red')
plt.title('Silhouette Score for K-Means')
plt.xlabel('Number of Clusters (k)')
plt.ylabel('Silhouette Score')
plt.grid(True)
plt.tight_layout()
plt.show()


optimal_k = 4
print(f"Optimal K chosen (visually): {optimal_k}")

kmeans_model = KMeans(n_clusters=optimal_k, random_state=42, n_init=10)
```

```
clusters_kmeans = kmeans_model.fit_predict(X_cluster_scaled)

print(f"K-Means cluster distribution (k={optimal_k}):\n{pd.Series(clusters_kmeans).value_counts().sort_index()}")
print(f"K-Means Silhouette Score: {silhouette_score(X_cluster_scaled, clusters_kmeans):.4f}")

# Visualize K-Means clusters (using PCA for 2D)
plt.figure(figsize=(8, 6))
sns.scatterplot(
    x=X_cluster_pca[:, 0], y=X_cluster_pca[:, 1], hue=clusters_kmeans,
    palette='viridis', legend='full', s=50, alpha=0.7
)
plt.title(f'K-Means Clusters (k={optimal_k}) - PCA Reduced')
plt.xlabel('Principal Component 1')
plt.ylabel('Principal Component 2')
plt.grid(True)
plt.show()

print("\n10.3 HIERARCHICAL CLUSTERING (AGGLOMERATIVE)")

linked = linkage(X_cluster_scaled, method='ward')

# Plot the dendrogram
plt.figure(figsize=(15, 7))
dendrogram(
    linked,
    orientation='top',
    distance_sort='descending',
    show_leaf_counts=True,
    color_threshold=linked[-optimal_k, 2] # Highlight selected clusters
)
plt.title('Hierarchical Clustering Dendrogram')
plt.xlabel('Sample Index or Cluster Size')
plt.ylabel('Distance')
plt.show()

agg_clustering_model = AgglomerativeClustering(n_clusters=optimal_k, linkage='ward')
clusters_agg = agg_clustering_model.fit_predict(X_cluster_scaled)

print(f"Hierarchical Cluster distribution (k={optimal_k}):\n{pd.Series(clusters_agg).value_counts().sort_index()}")
print(f"Hierarchical Silhouette Score: {silhouette_score(X_cluster_scaled, clusters_agg):.4f}")

# Visualize Hierarchical clusters (using PCA for 2D)
plt.figure(figsize=(8, 6))
sns.scatterplot(
    x=X_cluster_pca[:, 0], y=X_cluster_pca[:, 1], hue=clusters_agg,
    palette='viridis', legend='full', s=50, alpha=0.7
)
plt.title(f'Hierarchical Clusters (k={optimal_k}) - PCA Reduced')
plt.xlabel('Principal Component 1')
plt.ylabel('Principal Component 2')
plt.grid(True)
plt.show()
```